

Université Paris-Saclay
UVSQ - Université de Versailles Saint-Quentin-en-Yvelines

Projet de Calcul Scientifique

Méthode des Éléments Finis en 2D

Par Charbel Yannick ECLOU

Sous la supervision de **Tahar Boulmezaoud**
Professeur au Laboratoire de Mathématiques de Versailles

Master 1 Algèbre Appliquée

13 mai 2026

1 Préliminaires

On considère le domaine rectangulaire $\Omega =]-a, a[\times]-b, b[$ avec $a > 0$ et $b > 0$, ainsi que le problème aux dérivées partielles suivant :

$$-\frac{\partial}{\partial x} \left(\eta \frac{\partial u}{\partial x} \right) - \frac{\partial}{\partial y} \left(\eta \frac{\partial u}{\partial y} \right) + \lambda u = f \quad \text{dans } \Omega, \quad (1)$$

complété par les conditions homogènes :

$$\frac{\partial u}{\partial x}(-a, y) = 0, \quad \frac{\partial u}{\partial x}(a, y) = 0, \quad (2)$$

$$\frac{\partial u}{\partial y}(x, -b) = 0, \quad \frac{\partial u}{\partial y}(x, b) = 0. \quad (3)$$

On suppose que $\lambda > 0$, que $\eta \in C^1(\Omega)$ vérifie $\eta(x, y) \geq \eta_0 > 0$ et que f est continue sur Ω .

1.1 Formulation faible

Soit $v \in V$ une fonction test. On multiplie l'équation

$$-\nabla \cdot (\eta \nabla u) + \lambda u = f$$

par v et on intègre sur Ω :

$$\int_{\Omega} (-\nabla \cdot (\eta \nabla u) + \lambda u) v \, dx dy = \int_{\Omega} f v \, dx dy.$$

On traite séparément le premier terme en utilisant une intégration par parties (formule de Green) :

$$\int_{\Omega} -\nabla \cdot (\eta \nabla u) v \, dx dy.$$

En appliquant la formule de divergence, on obtient :

$$\int_{\Omega} -\nabla \cdot (\eta \nabla u) v \, dx dy = \int_{\Omega} \eta \nabla u \cdot \nabla v \, dx dy - \int_{\partial \Omega} v \eta \nabla u \cdot n \, d\sigma,$$

où n désigne le vecteur normal extérieur à $\partial \Omega$.

Ainsi, on peut écrire :

$$\int_{\Omega} (-\nabla \cdot (\eta \nabla u)) v \, dx dy = \int_{\Omega} \eta \nabla u \cdot \nabla v \, dx dy - \int_{\partial \Omega} \eta \frac{\partial u}{\partial n} v \, d\sigma.$$

Or, d'après les conditions homogènes, on a :

$$\frac{\partial u}{\partial n} = 0 \quad \text{sur } \partial\Omega.$$

Donc le terme de bord s'annule :

$$\int_{\partial\Omega} \eta \frac{\partial u}{\partial n} v \, d\sigma = 0.$$

On en déduit :

$$\int_{\Omega} -\nabla \cdot (\eta \nabla u) v \, dxdy = \int_{\Omega} \eta \nabla u \cdot \nabla v \, dxdy.$$

En reportant dans l'équation initiale, on obtient :

$$\int_{\Omega} \eta \nabla u \cdot \nabla v \, dxdy + \lambda \int_{\Omega} uv \, dxdy = \int_{\Omega} fv \, dxdy.$$

Finalement, la formulation faible du problème s'écrit :

$$(P) \quad \forall v \in V, \quad \int_{\Omega} (\eta \nabla u \cdot \nabla v + \lambda uv) \, dxdy = \int_{\Omega} fv \, dxdy.$$

1.2 Unicité de la solution

Soit $w \in V$ une autre solution vérifiant la même formulation faible :

$$\forall v \in V, \quad \int_{\Omega} (\eta \nabla w \cdot \nabla v + \lambda wv) \, dxdy = \int_{\Omega} fv \, dxdy.$$

On considère la différence $z = w - u$. En soustrayant les deux égalités, on obtient :

$$\forall v \in V, \quad \int_{\Omega} (\eta \nabla z \cdot \nabla v + \lambda zv) \, dxdy = 0.$$

En prenant $v = z$, il vient :

$$\int_{\Omega} \eta |\nabla z|^2 \, dxdy + \lambda \int_{\Omega} z^2 \, dxdy = 0.$$

Comme $\eta \geq \eta_0 > 0$ et $\lambda > 0$, les deux termes sont positifs. Ainsi :

$$\nabla z = 0 \quad \text{et} \quad z = 0,$$

d'où $z = 0$ dans Ω , ce qui implique $w = u$.

La solution du problème est donc unique.

1.3 Solution explicite dans un cas particulier

On suppose que :

$$a = b = 1, \quad \eta = 1, \quad \lambda = 1,$$

et

$$f(x, y) = \cos(m\pi x) \cos(m\pi y), \quad m \in N.$$

Le problème devient alors :

$$-\Delta u + u = \cos(m\pi x) \cos(m\pi y) \quad \text{dans } \Omega \text{ avec des conditions de Neumann homogènes.}$$

On cherche une solution sous la forme séparée :

$$u(x, y) = X(x)Y(y).$$

En substituant dans l'équation, on obtient :

$$-(X''(x)Y(y) + X(x)Y''(y)) + X(x)Y(y) = \cos(m\pi x) \cos(m\pi y).$$

On réécrit :

$$-X''(x)Y(y) - X(x)Y''(y) + X(x)Y(y) = \cos(m\pi x) \cos(m\pi y).$$

On cherche une solution adaptée au second membre, ce qui suggère de prendre :

$$X(x) = A \cos(m\pi x), \quad Y(y) = \cos(m\pi y),$$

où A est une constante à déterminer.

Calculons les dérivées :

$$X''(x) = -Am^2\pi^2 \cos(m\pi x), \quad Y''(y) = -m^2\pi^2 \cos(m\pi y).$$

On remplace dans l'équation :

$$-(-Am^2\pi^2 \cos(m\pi x) \cos(m\pi y) + A \cos(m\pi x)(-m^2\pi^2 \cos(m\pi y))) + A \cos(m\pi x) \cos(m\pi y).$$

On simplifie :

$$= Am^2\pi^2 \cos(m\pi x) \cos(m\pi y) + Am^2\pi^2 \cos(m\pi x) \cos(m\pi y) + A \cos(m\pi x) \cos(m\pi y).$$

$$= A(2m^2\pi^2 + 1) \cos(m\pi x) \cos(m\pi y).$$

En identifiant avec le second membre, on obtient :

$$A(2m^2\pi^2 + 1) = 1,$$

d'où :

$$A = \frac{1}{1 + 2m^2\pi^2}.$$

On en déduit la solution :

$$u_p(x, y) = \frac{1}{1 + 2m^2\pi^2} \cos(m\pi x) \cos(m\pi y).$$

Vérification des conditions aux limites.

On a :

$$\frac{\partial u_p}{\partial x}(x, y) = -\frac{m\pi}{1 + 2m^2\pi^2} \sin(m\pi x) \cos(m\pi y).$$

Donc :

$$\frac{\partial u_p}{\partial x}(\pm 1, y) = 0,$$

car $\sin(m\pi) = 0$.

De même :

$$\frac{\partial u_p}{\partial y}(x, \pm 1) = 0.$$

Les conditions homogènes sont donc satisfaites.

Ainsi, la solution explicite du problème est :

$$u_p(x, y) = \frac{1}{1 + 2m^2\pi^2} \cos(m\pi x) \cos(m\pi y).$$

1.4 Théorème de changement de variables pour les intégrales multiples

On rappelle le théorème de changement de variables pour les intégrales multiples.

Soient U et V deux ouverts de R^n et soit

$$F : U \rightarrow V$$

une application de classe C^1 , bijective, dont l'inverse est également de classe C^1 .

On note $J_F(x)$ la matrice jacobienne de F au point x , et $\det(J_F(x))$ son déterminant.

Alors, pour toute fonction intégrable $g : V \rightarrow R$, on a :

$$\int_V g(y) dy = \int_U g(F(x)) |\det(J_F(x))| dx.$$

2 Le maillage

2.1 Les nœuds

On considère désormais une discrétisation du domaine $\Omega =]-a, a[\times]-b, b[$.

Pour cela, on introduit une subdivision de l'intervalle $[-a, a]$:

$$x_0 = -a < x_1 < \cdots < x_N = a,$$

ainsi qu'une subdivision de l'intervalle $[-b, b]$:

$$y_0 = -b < y_1 < \cdots < y_M = b.$$

Ici, $N \geq 0$ et $M \geq 0$ sont deux entiers, supposés grands dans la pratique, représentant respectivement le nombre de subdivisions selon les directions x et y .

On obtient ainsi une grille de points du plan, appelés **nœuds du maillage**, définis par :

$$(x_i, y_j), \quad \text{pour } 0 \leq i \leq N, 0 \leq j \leq M.$$

Le nombre total de nœuds est donc égal à :

$$(N + 1)(M + 1).$$

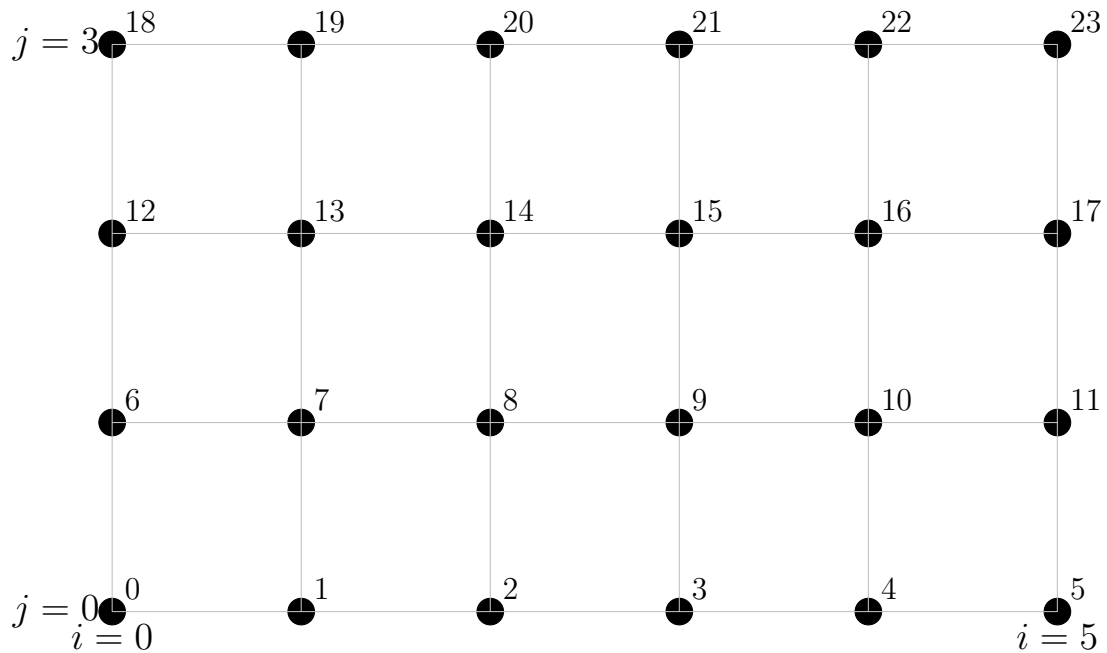
2.2 Illustration de la numérotation des nœuds

On considère ici un exemple avec $N = 5$ et $M = 3$. Le nombre total de nœuds est donc $(N + 1)(M + 1) = 6 \times 4 = 24$.

La numérotation des nœuds est définie par :

$$s_{i,j} = (N + 1)j + i.$$

La figure suivante illustre cette numérotation.



2.3 Question 5

Fonction, `Subdiv`, prenant en paramètres a et N , et renvoyant en sortie un vecteur $(x_i)_{0 \leq i \leq N}$ de taille $N + 1$, correspondant à une subdivision uniforme de l'intervalle $[-a, a]$ en $N + 1$ points.

```
// on vas faire une subdivision uniforme de [-a, a] en N+1
points
vector<double> Subdiv(double a, int N){
    //on cree un vecteur de taille N+1 contenant les points
    de subdivision
    vector<double> xi(N+1);
    for(int i = 0 ; i<=N ; i++){
        // xi[i] = -a + i * pas, avec pas = 2a/N
        xi[i] = -a + i * (2*a) / N;
    }
    return xi;
}
```

2.4 Question 7

Fonction, `numgb`, permettant d'associer à un couple d'indices (i, j) , avec $0 \leq i \leq N$ et $0 \leq j \leq M$, un indice global k correspondant à la numérotation des nœuds du maillage.

```
// numero global d'un noeud (i,j) dans la grille
// on applique la formule : s = (N+1)*j + i
int numgb(int N, int M, int i, int j){
```

```

    return (N+1)*j + i;
}

```

Question 8. Montrons qu'il existe un seul couple (i, j) , $0 \leq i \leq N$ et $0 \leq j \leq M$, tel que $S_{i,j} = S$. On effectue la division euclidienne de S par $N + 1$.

$$S = (N + 1) \cdot j + i \quad \text{avec} \quad 0 \leq i \leq N$$

Par le théorème de la division euclidienne, cette décomposition existe et est unique. On obtient :

$$i \equiv S \pmod{N + 1}, \quad j = \left\lfloor \frac{S}{N + 1} \right\rfloor$$

Vérifions que $j \leq M$: Puisque $S < (N + 1)(M + 1) - 1$, on a :

$$j = \left\lfloor \frac{S}{N + 1} \right\rfloor \leq \left\lfloor \frac{(N + 1)(M + 1) - 1}{N + 1} \right\rfloor = M$$

Car $(N + 1)(M + 1) - 1 < (N + 1)(M + 1)$.

Question 9. Fonction `invnumgb`, qui, à partir d'un indice global k , permet de retrouver le couple d'indices (i, j) associé.

```

// On inverse de numgb : retrouve (i,j) depuis le numero
// global S
// i = S mod (N+1) et j = S / (N+1)
vector<int> invnumgb(int N, int M, int S){
    int i = S % (N+1); // reste de la division euclidienne =
        indice en x
    int j = S / (N+1); // quotient de la division euclidienne
        = indice en y
    return {i, j};
}

```

2.5 Triangulation du maillage

On construit maintenant la famille de $2NM$ triangles définis de la manière suivante : dans chacun des rectangles $[x_i, x_{i+1}] \times [y_j, y_{j+1}]$, $0 \leq i \leq N - 1$, $0 \leq j \leq M - 1$, on construit les deux triangles $T_{i,j}^-$ et $T_{i,j}^+$ en alternant horizontalement et verticalement les choix suivants :

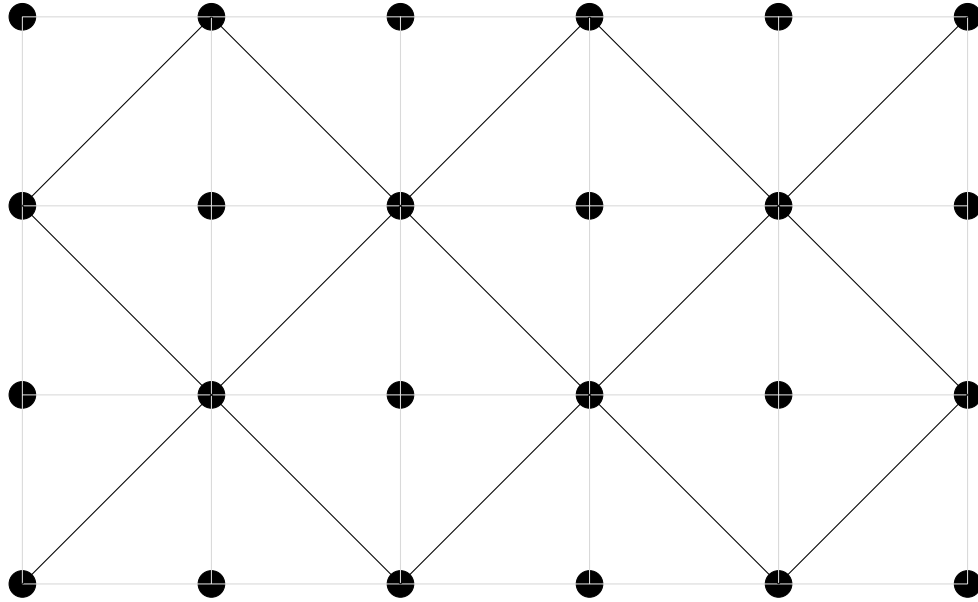
- $T_{i,j}^-$ le triangle de sommets (x_i, y_j) , (x_{i+1}, y_j) , (x_{i+1}, y_{j+1}) et $T_{i,j}^+$ le triangle de sommets (x_i, y_j) , (x_i, y_{j+1}) et (x_{i+1}, y_{j+1}) ,
ou
- $T_{i,j}^-$ le triangle de sommets (x_i, y_j) , (x_{i+1}, y_j) , (x_i, y_{j+1}) et $T_{i,j}^+$ le triangle de sommets (x_{i+1}, y_j) , (x_i, y_{j+1}) et (x_{i+1}, y_{j+1}) .

On a ainsi un ensemble $\mathcal{T}$ de K triangles, avec $K = 2NM$. On conviendra qu'un triangle T sera identifié à un triplet (n_1, n_2, n_3) où n_1 , n_2 et n_3 désignent les numéros des trois sommets de T .

On numérotera ces triangles ligne par ligne de gauche à droite, de bas en haut en commençant par 0. On posera dans la suite :

$$\mathcal{T} = \{T_\ell \mid 0 \leq \ell \leq K - 1\},$$

où T_ℓ est le ℓ -ième triangle. Ainsi, $\mathcal{T}$ représente une triangulation de Ω .



2.6 Question 11

Fonction `maillageTR` ayant comme paramètres les entiers N, M et renvoyant un tableau (ou matrice) TRG de taille $K \times 3$ dont la ℓ -ème ligne, $0 \leq \ell \leq K - 1$, contient les trois numéros des sommets de T_ℓ .

```
//On fait la construction du maillage triangulaire
// On vas retourne une matrice TRG de taille K x 3, K = 2*N*M
// triangles
// Chaque ligne va contenir les 3 indices globaux des sommets
vector<vector<int>> maillageTR(int N, int M) {
    // K = nombre total de triangles
    int K = 2 * N * M;
    // TRG : table de connectivite
    vector<vector<int>> TRG(K);
```

```

// l : indice du triangle courant
int l = 0;
// parcours de chaque cellule rectangulaire (i,j)
for (int j = 0; j < M; j++) {
    for (int i = 0; i < N; i++) {
        // indices globaux des 4 coins de la cellule
        int s1 = numgb(N,M,i, j ); // bas-gauche
        int s2 = numgb(N,M,i+1,j ); // bas-droite
        int s3 = numgb(N,M,i, j+1); // haut-gauche
        int s4 = numgb(N,M,i+1,j+1); // haut-droite
        // decoupage en 2 triangles selon la parite de i+j
        if ((i + j) % 2 == 0) {
            // diagonale de s2 vers s3 (bas-droite -> haut-gauche)
            TRG[l++] = {s1, s2, s4}; // triangle T-
            TRG[l++] = {s1, s3, s4}; // triangle T+
        } else {
            // diagonale de s1 vers s4 (bas-gauche -> haut-droite)
            TRG[l++] = {s1, s2, s3}; // triangle T-
            TRG[l++] = {s2, s3, s4}; // triangle T+
        }
    }
}
return TRG;
}

```

2.6.1 Coordonnées barycentriques dans un triangle. Triangle de référence.

On dit qu'une application f de R^n dans R^m , $n \geq 1$ et $m \geq 1$, est affine si

$$\forall \vec{x}, \vec{y} \in R^n, \forall \alpha \in R, f(\alpha \vec{x} + (1 - \alpha) \vec{y}) = \alpha f(\vec{x}) + (1 - \alpha) f(\vec{y}).$$

Soient A_0 , A_1 et A_2 trois points non alignés du plan euclidien R^2 . On note T le triangle de sommets A_0 , A_1 et A_2 . Ainsi, T est non aplati.

Les coordonnées du point A_j , $0 \leq j \leq 2$, seront notées (x_j, y_j) .

Question 12. Montrer qu'une application f de R^n dans R^m est affine si et seulement si l'application

$$\vec{x} \mapsto f(\vec{x}) - f(0)$$

est linéaire.

Soit g l'application $\vec{x} \mapsto f(\vec{x}) - f(0)$.

($\Leftarrow$) **On suppose g linéaire.** Pour tout $\vec{x}, \vec{y} \in R^n$ et $\alpha \in R$:

$$\begin{aligned} f(\alpha\vec{x} + (1 - \alpha)\vec{y}) &= g(\alpha\vec{x} + (1 - \alpha)\vec{y}) + f(0) \\ &= \alpha g(\vec{x}) + (1 - \alpha)g(\vec{y}) + f(0) \quad \text{par linéarité de } g. \end{aligned}$$

Or, comme $g(\vec{x}) = f(\vec{x}) - f(0)$, on a :

$$\begin{aligned} f(\alpha\vec{x} + (1 - \alpha)\vec{y}) &= \alpha(f(\vec{x}) - f(0)) + (1 - \alpha)(f(\vec{y}) - f(0)) + f(0) \\ &= \alpha f(\vec{x}) - \alpha f(0) + (1 - \alpha)f(\vec{y}) - (1 - \alpha)f(0) + f(0) \\ &= \alpha f(\vec{x}) + (1 - \alpha)f(\vec{y}) \end{aligned}$$

donc f est affine.

($\Rightarrow$) **Supposons f est affine.** Pour $\vec{y} = 0$:

$$\begin{aligned} f(\alpha\vec{x}) &= f(\alpha\vec{x} + (1 - \alpha)\vec{y}) \\ &= \alpha f(\vec{x}) + (1 - \alpha)f(\vec{y}) \\ &= \alpha f(\vec{x}) + f(\vec{y}) - \alpha f(\vec{y}) \\ &= \alpha f(\vec{x}) + f(0) - \alpha f(0) \end{aligned}$$

$$\begin{aligned} f(\alpha\vec{x}) - f(0) &= \alpha(f(\vec{x}) - f(0)) \\ g(\alpha\vec{x}) &= \alpha g(\vec{x}) \quad (1) \text{ donc on a l'homogénéité.} \end{aligned}$$

Pour $\alpha = \frac{1}{2}$:

$$f\left(\frac{\vec{x} + \vec{y}}{2}\right) = \frac{1}{2}f(\vec{x}) + \frac{1}{2}f(\vec{y})$$

$$g\left(\frac{\vec{x} + \vec{y}}{2}\right) = g\left(\frac{1}{2}(\vec{x} + \vec{y})\right) = \frac{1}{2}g(\vec{x} + \vec{y}) \quad \text{d'après (1).}$$

$$\begin{aligned} g(\vec{x} + \vec{y}) &= 2g\left(\frac{\vec{x} + \vec{y}}{2}\right) \\ &= 2\left(f\left(\frac{\vec{x} + \vec{y}}{2}\right) - f(0)\right) \\ &= 2\left(\frac{1}{2}f(\vec{x}) + \frac{1}{2}f(\vec{y}) - f(0)\right) \\ &= f(\vec{x}) + f(\vec{y}) - 2f(0) \\ &= (f(\vec{x}) - f(0)) + (f(\vec{y}) - f(0)) \\ &= g(\vec{x}) + g(\vec{y}) \end{aligned}$$

D'où g est linéaire.

Question 13 Dédudions $f : R^2 \rightarrow R$ affine $\iff f(x, y) = \alpha x + \beta y + \gamma$

($\Leftarrow$) Supposons que $f(x, y) = \alpha x + \beta y + \gamma$.

$g(x, y) = f(x, y) - f(0, 0) = \alpha x + \beta y$, qui est linéaire. (D'après la question 12), alors f est affine.

($\Rightarrow$) Supposons f est affine.

$g(x, y) = f(x, y) - f(0, 0)$ est linéaire de R^2 dans R . Donc elle peut s'écrire sous la forme :

$$g(x, y) = g(x \cdot e_1 + y \cdot e_2) = xg(e_1) + yg(e_2)$$

En posant : $\alpha = g(1, 0)$, $\beta = g(0, 1)$ et $\gamma = f(0, 0)$.

On obtient :

$$\begin{aligned} f(x, y) &= g(x, y) + f(0, 0) \\ &= xg(1, 0) + yg(0, 1) + f(0, 0) \\ f(x, y) &= \alpha x + \beta y + \gamma \end{aligned}$$

D'où le résultat.

Question 14 Montrer que pour tout point $M(x, y)$ du plan, il existe trois réels $\lambda_0(x, y)$, $\lambda_1(x, y)$ et $\lambda_2(x, y)$, uniques, tels que :

$$\begin{pmatrix} x \\ y \end{pmatrix} = \sum_{j=0}^2 \lambda_j \begin{pmatrix} x_j \\ y_j \end{pmatrix} \quad \text{et} \quad \sum_{j=0}^2 \lambda_j = 1.$$

On appellera $(\lambda_0(x, y), \lambda_1(x, y), \lambda_2(x, y))$ les coordonnées barycentriques de $M(x, y)$ par rapport à T .

($\Rightarrow$) On cherche les coordonnées $\lambda_0, \lambda_1, \lambda_2$ tels que $\sum \lambda_j x_j = x$ et $\sum \lambda_j y_j = y$. On obtient le système suivant :

$$\begin{pmatrix} 1 & 1 & 1 \\ x_0 & x_1 & x_2 \\ y_0 & y_1 & y_2 \end{pmatrix} \begin{pmatrix} \lambda_0 \\ \lambda_1 \\ \lambda_2 \end{pmatrix} = \begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$$

On a :

$$M \begin{pmatrix} \lambda_0 \\ \lambda_1 \\ \lambda_2 \end{pmatrix} = \begin{pmatrix} 1 \\ x \\ y \end{pmatrix}$$

D'autre part on a :

$$\begin{aligned}\det(M) &= (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0) \\ &= 2|T| \neq 0\end{aligned}$$

Car A_0, A_1, A_2 sont non alignés. Le système est donc inversible et admet une unique solution pour tout (x, y) .

15) Les λ_i sont affines : Par la règle de Cramer, chaque $\lambda_i(x, y)$ s'exprime comme un rapport de déterminants dont le numérateur est linéaire en $(x, y, 1)$. D'après la question 13, c'est donc une fonction de la forme $ax + by + \gamma$: d'où chaque $\lambda_i(x, y)$ est affine.

16) La valeur de $\lambda_j(x_i, y_i)$ pour $0 \leq i, j \leq 2$: Le sommet $A_i = (x_i, y_i)$ a pour coordonnées barycentriques le vecteur unité e_i . Donc :

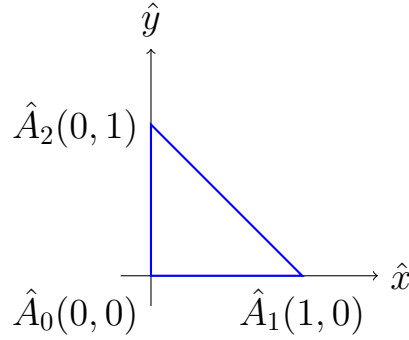
$$\lambda_j(x_i, y_i) = \delta_{i,j} = \begin{cases} 1 & \text{si } i = j \\ 0 & \text{si } i \neq j \end{cases}$$

17) Montrons que toute fonction affine f s'écrit $f = \sum_{i=0}^2 f(x_i, y_i)\lambda_i$: Posons $h = f - \sum_{i=0}^2 f(x_i, y_i)\lambda_i$. Cette fonction h est affine car les f est affine et λ_i affine. Pour $k = 0, 1, 2$:

$$\begin{aligned}h(x_k, y_k) &= f(x_k, y_k) - \sum_{i=0}^2 f(x_i, y_i)\lambda_i(x_k, y_k) \\ &= f(x_k, y_k) - \sum_{i=0}^2 f(x_i, y_i)\delta_{i,k} \\ &= f(x_k, y_k) - f(x_k, y_k) = 0\end{aligned}$$

donc h est affine et s'annule en trois points non alignés. Par la question 13, $h = \alpha x + \beta y + \gamma$ s'annulant en trois points non alignés implique $\alpha = \beta = \gamma = 0$, donc $h \equiv 0$. D'où $f = \sum_{i=0}^2 f(x_i, y_i)\lambda_i$.

18) Triangle de référence On note $\hat{T}$ le triangle de référence dont les sommets sont les points $\hat{A}_i, 0 \leq i \leq 2$ de coordonnées $(\hat{x}_0, \hat{y}_0) = (0, 0)$, $(\hat{x}_1, \hat{y}_1) = (1, 0)$ et $(\hat{x}_2, \hat{y}_2) = (0, 1)$ et $\hat{\lambda}_0, \hat{\lambda}_1$ et $\hat{\lambda}_2$ les coordonnées barycentriques par rapport à ce triangle.



Fonctions barycentriques : $\hat{\lambda}_0(x, y) = 1 - x - y$

$$\hat{\lambda}_1(x, y) = x$$

$$\hat{\lambda}_2(x, y) = y$$

Vérification :

$$\hat{\lambda}_0(0, 0) = 1, \quad \hat{\lambda}_1(0, 0) = 0, \quad \hat{\lambda}_2(0, 0) = 0$$

$$\hat{\lambda}_0(1, 0) = 0, \quad \hat{\lambda}_1(1, 0) = 1, \quad \hat{\lambda}_2(1, 0) = 0$$

$$\hat{\lambda}_0(0, 1) = 0, \quad \hat{\lambda}_1(0, 1) = 0, \quad \hat{\lambda}_2(0, 1) = 1$$

19) Existence et unicité de F_T On cherche $F_T : R^2 \rightarrow R^2$ affine telle que $F_T(\hat{A}_i) = A_i$ pour $i = 0, 1, 2$.

D'après la question 17, une application affine est entièrement déterminée par ses valeurs en trois points non alignés. Les $\hat{A}_i$ sont non alignés, donc F_T est unique.

$$F_T(\hat{x}, \hat{y}) = \sum_{i=0}^2 \hat{\lambda}_i(\hat{x}, \hat{y}) A_i = (1 - \hat{x} - \hat{y}) A_0 + \hat{x} A_1 + \hat{y} A_2$$

donc

$$F_T \begin{pmatrix} \hat{x} \\ \hat{y} \end{pmatrix} = \begin{pmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{pmatrix} \begin{pmatrix} \hat{x} \\ \hat{y} \end{pmatrix} + \begin{pmatrix} x_0 \\ y_0 \end{pmatrix}$$

20) Montrons que F_T est inversible On sait que la partie linéaire de

F_T est $B_T = \begin{pmatrix} x_1 - x_0 & x_2 - x_0 \\ y_1 - y_0 & y_2 - y_0 \end{pmatrix}$ et $\det B_T = (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0) = 2|T| \neq 0$ car T est non aplati. Donc B_T est inversible.

Ainsi F_T est bijective d'inverse :

$$F_T^{-1}(x, y) = B_T^{-1} \begin{pmatrix} x - x_0 \\ y - y_0 \end{pmatrix}$$

22) Fonction CalcMatBT

- ayant comme paramètres deux vecteurs (ou tableaux) $\mathbf{xs}$ et $\mathbf{ys}$ comportant les coordonnées des trois sommets d'un triangle T
- qui renvoie la matrice B_T de taille 2×2 du triangle T .

```
// Calcule la matrice jacobienne B_T du triangle T
vector<vector<double>> CalcMatBT(vector<double> xs, vector<
double> ys) {
    // BT est une matrice 2x2
    vector<vector<double>> BT(2, vector<double>(2));
    // premiere ligne : differences en x
    BT[0] = {xs[1] - xs[0], xs[2] - xs[0]};
    // deuxieme ligne : differences en y
    BT[1] = {ys[1] - ys[0], ys[2] - ys[0]};
    return BT;
}
```

23) Exprimons l'aire de T en fonction de $\det(B_T)$. On a :

$$|\det(B_T)| = |(x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0)| = 2 \text{Aire}(T)$$

D'où :

$$|T| = \frac{1}{2} |\det(B_T)|$$

24) Expression de λ_i en fonction de $\hat{\lambda}_i$ et F_T . On sait que λ_i est l'unique fonction affine sur T valant δ_{ij} au sommet A_j . Or $\hat{\lambda}_i \circ F_T^{-1}$ est affine et vérifie :

$$(\hat{\lambda}_i \circ F_T^{-1})(A_j) = \hat{\lambda}_i(\hat{A}_j) = \delta_{ij}$$

Par unicité :

$$\lambda_i = \hat{\lambda}_i \circ F_T^{-1}$$

25) En déduire l'expression de $\nabla \lambda_i$, $0 \leq i \leq 2$, en fonction de la matrice B_T et du vecteur $\nabla \hat{\lambda}_i$. De $\lambda_i = \hat{\lambda}_i \circ F_T^{-1}$, on a :

$$\nabla \lambda_i(x, y) = (DF_T^{-1})^T \cdot \nabla \hat{\lambda}_i(F_T^{-1}(x, y))$$

Or $DF_T^{-1} = B_T^{-1}$ car F_T est affine de partie linéaire B_T . De plus $\nabla \hat{\lambda}_i$ est constante car $\hat{\lambda}_i$ est affine, donc :

$$\boxed{\nabla \lambda_i = B_T^{-T} \nabla \hat{\lambda}_i}$$

26) Soient k, j deux entiers tels que $0 \leq k, j \leq 2$. On effectue le changement de variable $(x, y) = F_T(\hat{x}, \hat{y})$:

$$\int_T \eta \nabla \lambda_k \cdot \nabla \lambda_j dx dy = (\nabla \lambda_k \cdot \nabla \lambda_j) \int_T \eta dx dy$$

car $\nabla \lambda_k$ et $\nabla \lambda_j$ sont constantes sur T . En utilisant $\nabla \lambda_i = B_T^{-T} \nabla \hat{\lambda}_i$:

$$\begin{aligned} \nabla \lambda_k \cdot \nabla \lambda_j &= (B_T^{-T} \nabla \hat{\lambda}_k)^T (B_T^{-T} \nabla \hat{\lambda}_j) \\ &= \nabla \hat{\lambda}_k^T (B_T^{-1} B_T^{-T}) \nabla \hat{\lambda}_j \\ &= \nabla \hat{\lambda}_k^T (B_T^T B_T)^{-1} \nabla \hat{\lambda}_j \end{aligned}$$

Et $\int_T \eta dx dy = |\det B_T| \int_{\hat{T}} \eta(F_T(\hat{x}, \hat{y})) d\hat{x} d\hat{y}$. Donc :

$$\int_T \eta \nabla \lambda_k \cdot \nabla \lambda_j dx dy = \nabla \hat{\lambda}_k^T (B_T^T B_T)^{-1} \nabla \hat{\lambda}_j \times |\det B_T| \int_{\hat{T}} \eta(F_T(\hat{x}, \hat{y})) d\hat{x} d\hat{y}$$

27) Expressions de $\int_{\hat{T}} \hat{\lambda}_k \hat{\lambda}_j d\hat{x} d\hat{y}$. On utilise la formule exacte d'intégration des monômes sur $\hat{T}$:

$$\int_{\hat{T}} \hat{x}^p \hat{y}^q d\hat{x} d\hat{y} = \frac{p!q!}{(p+q+2)!}$$

On rappelle $\hat{\lambda}_0 = 1 - \hat{x} - \hat{y}$, $\hat{\lambda}_1 = \hat{x}$, $\hat{\lambda}_2 = \hat{y}$.

— Cas diagonale ($k = j$) :

— $k = j = 1$: $\int_{\hat{T}} \hat{x}^2 d\hat{x} d\hat{y} = \frac{2!0!}{4!} = \frac{2}{24} = \frac{1}{12}$

— $k = j = 2$: $\int_{\hat{T}} \hat{y}^2 d\hat{x} d\hat{y} = \frac{0!2!}{4!} = \frac{1}{12}$

— $k = j = 0$: $\int_{\hat{T}} (1 - \hat{x} - \hat{y})^2 d\hat{x} d\hat{y} = \frac{1}{12}$ (par symétrie)

— Cas hors-diagonale ($k \neq j$) :

— $k = 1, j = 2$: $\int_{\hat{T}} \hat{x} \hat{y} d\hat{x} d\hat{y} = \frac{1!1!}{4!} = \frac{1}{24}$

— $k = 0, j = 1$: $\int_{\hat{T}} (1 - \hat{x} - \hat{y}) \hat{x} d\hat{x} d\hat{y} = \int_{\hat{T}} (\hat{x} - \hat{x}^2 - \hat{x} \hat{y}) d\hat{x} d\hat{y} = \frac{1}{6} - \frac{1}{12} - \frac{1}{24} = \frac{1}{24}$

Ainsi :

$$\int_{\hat{T}} \hat{\lambda}_k \hat{\lambda}_j d\hat{x} d\hat{y} = \begin{cases} \frac{1}{12} & \text{si } k = j \\ \frac{1}{24} & \text{si } k \neq j \end{cases}$$

28) On s'intéresse maintenant au calcul des intégrales

$$\int_T \eta(x, y) dx dy.$$

Pour calculer cette intégrale, on peut effectuer le changement de variable $(x, y) = F_T(x, y)$ pour la ramener à $\hat{T}$. Ainsi, la formule de changement de variables donne

$$\int_T \eta(x, y) dx dy = |\det(B_T)| \int_{\hat{T}} \eta(F_T(x, y)) dx dy.$$

On est donc amené à calculer des intégrales de la forme

$$\int_{\hat{T}} h(x, y) dx dy$$

On aimerait utiliser une formule de quadrature du type

$$\int_{\hat{T}} h(x, y) dx dy = \int_0^1 \int_0^{1-x} h(x, y) dy dx = \int_0^1 \int_0^{1-y} h(x, y) dx dy \approx \sum_{i=1}^q \omega_i h(x_i, y_i),$$

où (x_i, y_i) , $1 \leq i \leq q$, sont des points et ω_i les poids associés, tous choisis convenablement.

Par exemple, on a la formule de quadrature des points milieux :

$$\int_{\hat{T}} h(x, y) dx dy \approx \frac{1}{6} \left[h\left(\frac{1}{2}, 0\right) + h\left(0, \frac{1}{2}\right) + h\left(\frac{1}{2}, \frac{1}{2}\right) \right],$$

ou la formule

$$\int_{\hat{T}} h(x, y) dx dy \approx \frac{1}{6} \left[h\left(\frac{1}{6}, \frac{1}{6}\right) + h\left(\frac{1}{6}, \frac{2}{3}\right) + h\left(\frac{2}{3}, \frac{1}{6}\right) \right].$$

D'autres formules de quadrature sur le triangle de référence peuvent être proposées et utilisées.

28 (a) On cherche à déterminer le degré de précision des formules de quadrature proposées sur le triangle de référence $\hat{T}$.

Rappel. Une formule de quadrature est dite exacte pour les polynômes de degré inférieur ou égal à d si elle donne la valeur exacte de l'intégrale pour toute fonction de la forme

$$(x, y) \mapsto x^p y^q \quad \text{avec } p + q \leq d.$$

1. Formule des points milieux.

La formule est donnée par :

$$\int_{\hat{T}} h(x, y) dx dy \approx \frac{1}{6} \left[h\left(\frac{1}{2}, 0\right) + h\left(0, \frac{1}{2}\right) + h\left(\frac{1}{2}, \frac{1}{2}\right) \right].$$

Testons cette formule :

— Pour $h(x, y) = 1$:

$$\int_{\hat{T}} 1 \, dx \, dy = \frac{1}{2}, \quad \text{et} \quad \frac{1}{6}(1 + 1 + 1) = \frac{1}{2}.$$

Exact.

— Pour $h(x, y) = x$:

$$\int_{\hat{T}} x \, dx \, dy = \frac{1}{6}, \quad \text{et} \quad \frac{1}{6} \left(\frac{1}{2} + 0 + \frac{1}{2} \right) = \frac{1}{6}.$$

Exact.

— Pour $h(x, y) = x^2$:

$$\int_{\hat{T}} x^2 \, dx \, dy = \frac{1}{12}, \quad \text{mais la formule donne} \quad \frac{1}{6} \left(\frac{1}{4} + 0 + \frac{1}{4} \right) = \frac{1}{12}.$$

Exact.

— Pour $h(x, y) = xy$:

$$\int_{\hat{T}} xy \, dx \, dy = \frac{1}{24}, \quad \text{et la formule donne} \quad \frac{1}{6} \left(0 + 0 + \frac{1}{4} \right) = \frac{1}{24}.$$

Exact.

Cependant, pour des polynômes de degré 3, la formule n'est plus exacte.

Conclusion : cette formule est exacte pour les polynômes de degré $d = 2$.

2. Formule à trois points barycentriques.

La formule est :

$$\int_{\hat{T}} h(x, y) \, dx \, dy \approx \frac{1}{6} \left[h \left(\frac{1}{6}, \frac{1}{6} \right) + h \left(\frac{1}{6}, \frac{2}{3} \right) + h \left(\frac{2}{3}, \frac{1}{6} \right) \right].$$

On vérifie de la même manière que cette formule est exacte pour tous les polynômes de degré inférieur ou égal à 2, mais pas pour le degré 3.

28 (b). Fonction `integetatriang` ayant comme paramètre d'entrée une fonction `etacoeef` représentant η (ainsi que d'autres paramètres d'entrée nécessaires pour le calcul), et renvoyant à la sortie un vecteur `ET` de taille K (nombre des triangles du maillage) contenant les intégrales

$$\int_T \eta(x, y) \, dx \, dy$$

pour tous les triangles du maillage (ou plutôt des approximations de ces intégrales puisque les formules de quadrature ne sont pas toujours exactes).

```

// Calcule ET[l] = integrale de eta sur le triangle T_l
// par la formule de quadrature des points milieux
// ET[l] = |det(B_T)| * (1/6) * sum eta(points milieux)
vector<double> integ_eta_triangu(
    double (*eta_coef)(double, double),
    const vector<vector<int>>& TRG,
    const vector<double>& xs,
    const vector<double>& ys)
{
    // K = nombre de triangles
    int K = TRG.size();
    // ET[l] contiendra l'integrale de eta sur le triangle l
    vector<double> ET(K);

    for (int l = 0; l < K; l++) {
        // On recupere les indices globaux des 3 sommets
        int n0 = TRG[l][0], n1 = TRG[l][1], n2 = TRG[l][2];
        // coordonnees des 3 sommets
        double x0 = xs[n0], y0 = ys[n0];
        double x1 = xs[n1], y1 = ys[n1];
        double x2 = xs[n2], y2 = ys[n2];
        // determinant de B_T = 2 * aire du triangle
        double detBT = (x1-x0)*(y2-y0) - (x2-x0)*(y1-y0);
        // 1er point de quadrature : milieu du cote [A0,A1]
        double px1 = x0 + 0.5*(x1-x0);
        double py1 = y0 + 0.5*(y1-y0);
        // 2eme point de quadrature : milieu du cote [A0,A2]
        double px2 = x0 + 0.5*(x2-x0);
        double py2 = y0 + 0.5*(y2-y0);
        // 3eme point de quadrature : milieu du cote [A1,A2]
        double px3 = x0 + 0.5*(x1-x0) + 0.5*(x2-x0);
        double py3 = y0 + 0.5*(y1-y0) + 0.5*(y2-y0);
        // formule des points milieux : poids 1/6 pour chaque
        // point
        double quadrature = (1.0/6.0) * (eta_coef(px1,py1)
                                         + eta_coef(px2,py2)
                                         + eta_coef(px3,py3));
        // changement de variable : multiplication par |det(B_T)|
        ET[l] = abs(detBT) * quadrature;
    }
    return ET;
}

```

28 (c). Deux fonctions DiffTerm et ReacTerm

- ayant comme paramètres deux vecteurs (ou tableaux) **xs** et **ys** comportant les coordonnées des trois sommets d'un triangle T et éven-

tuellement un réel `val` représentant la valeur de l'intégrale

$$\int_T \eta(x, y) dx dy,$$

— et qui renvoie chacune un tableau de taille 6 représentant la matrice symétrique de taille 3×3 dont les coefficients sont, respectivement,

$$\int_T \eta \nabla \lambda_k \cdot \nabla \lambda_j dx dy,$$

$$\int_T \lambda_k \lambda_j dx dy, \quad 0 \leq k, j \leq 2,$$

où λ_k , $0 \leq k \leq 2$, désignent les coordonnées barycentriques par rapport à T .

```
// Calcule la matrice de reaction locale 3x3
// R[i][r] = |det(B_T)| * int_T_hat lambda_hat_i *
//           lambda_hat_r
// = |det(B_T)| * { 1/12 si i==r, 1/24 sinon }
vector<vector<double>> ReactTerm(vector<double> xs, vector<
double> ys) {
    // determinant de B_T
    double det = (xs[1]-xs[0])*(ys[2]-ys[0])
                - (xs[2]-xs[0])*(ys[1]-ys[0]);
    // valeur absolue du determinant
    double absdet = abs(det);
    // matrice de reaction 3x3
    vector<vector<double>> R(3);
    // diagonale : absdet/12, hors-diagonale : absdet/24
    R[0] = {absdet/12.0, absdet/24.0, absdet/24.0};
    R[1] = {absdet/24.0, absdet/12.0, absdet/24.0};
    R[2] = {absdet/24.0, absdet/24.0, absdet/12.0};
    return R;
}
// D[i][r] = val * (grad_lambda_i . grad_lambda_r)
// ou val = ET[t] = integrale de eta sur T
vector<vector<double>> DiffTerm(vector<double> xs, vector<
double> ys, double val) {
    // coefficients de B_T
    double a = xs[1]-xs[0], b = xs[2]-xs[0];
    double c = ys[1]-ys[0], d = ys[2]-ys[0];
    // determinant de B_T
    double det = a*d - b*c;
    // inverse de B_T : B_T^{-1} = (1/det) * | d  -b |
    //                                           | -c  a |
    double inv[2][2] = {{ d/det, -b/det},
                        {-c/det,  a/det}};
```

```

// gradients des fonctions de base sur T_hat
// grad_hat_lambda_0 = (-1,-1), grad_hat_lambda_1 = (1,0)
// grad_hat_lambda_2 = (0,1)
double gh[3][2] = {{-1,-1},{1,0},{0,1}};
// gradients sur T via g[i] = B_T^{-T} * grad_hat[i]
double g[3][2];
for (int i = 0; i < 3; i++) {
    // composante x du gradient transforme
    g[i][0] = inv[0][0]*gh[i][0] + inv[1][0]*gh[i][1];
    // composante y du gradient transforme
    g[i][1] = inv[0][1]*gh[i][0] + inv[1][1]*gh[i][1];
}
// matrice de diffusion 3x3
vector<vector<double>> D(3, vector<double>(3));
// D[i][j] = val * produit scalaire des gradients
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        D[i][j] = val * (g[i][0]*g[j][0] + g[i][1]*g[j][1]);
return D;
}

```

3 Le Problème discret

On considère maintenant la triangulation $\mathcal{T}$ de Ω ci-dessus (comportant $K = 2NM$ triangles). On pose $G = (N+1)(M+1)$. On note $M_0, M_1, \dots, M_{G-1}$ les noeuds du maillage. Soit V_h l'espace suivant

$$V_h = \{v \in C^0(\Omega) \mid \forall 0 \leq \ell \leq K-1, v|_{T_\ell} \text{ est affine}\},$$

où $C^0(\Omega)$ est l'espace des fonctions continues sur Ω .

28 (d). On considère l'application $I_h : V_h \rightarrow R^G$ définie par

$$\forall v \in V_h, \quad I_h(v) = (v(M_k))_{0 \leq k \leq G-1}.$$

1. Linéarité.

Soient $v, w \in V_h$ et $\alpha \in R$. On a :

$$I_h(v + \alpha w) = ((v + \alpha w)(M_k))_{0 \leq k \leq G-1} = (v(M_k) + \alpha w(M_k))_{0 \leq k \leq G-1}.$$

Donc :

$$I_h(v + \alpha w) = I_h(v) + \alpha I_h(w),$$

ce qui montre que I_h est linéaire.

2. Injectivité.

Soit $v \in V_h$ tel que $I_h(v) = 0$. Alors :

$$\forall k, \quad v(M_k) = 0.$$

Or, sur chaque triangle T_ℓ , la fonction v est affine et entièrement déterminée par ses valeurs aux trois sommets.

Comme v est nulle en tous les nœuds du maillage, elle est nulle sur chaque triangle, donc :

$$v \equiv 0 \quad \text{sur } \Omega.$$

Ainsi, I_h est injective.

3. Surjectivité.

Soit $(a_k)_{0 \leq k \leq G-1} \in R^G$. On construit une fonction $v \in V_h$ telle que :

$$v(M_k) = a_k.$$

Sur chaque triangle T_ℓ , on définit v comme l'unique fonction affine prenant les valeurs a_k aux sommets.

Cette définition est cohérente car les fonctions sont continues aux nœuds (valeurs imposées identiques).

Ainsi, $v \in V_h$ et $I_h(v) = (a_k)$, donc I_h est surjective.

Conclusion.

L'application I_h est linéaire, injective et surjective, donc c'est un isomorphisme de V_h sur R^G .

Dimension de V_h .

Comme $V_h \simeq R^G$, on en déduit :

$$\dim(V_h) = G = (N + 1)(M + 1).$$

28 (e). Pour tout entier k , $0 \leq k \leq G - 1$, on note w_k l'unique élément de V_h vérifiant

$$w_k(M_\ell) = \delta_{k,\ell}, \quad \text{pour tout } 0 \leq \ell \leq G - 1.$$

Montrons que la famille $(w_k)_{0 \leq k \leq G-1}$ est une base de V_h .

1. Génération.

Soit $v \in V_h$. Posons

$$v_h = \sum_{k=0}^{G-1} v(M_k) w_k.$$

Pour tout ℓ , on a :

$$v_h(M_\ell) = \sum_{k=0}^{G-1} v(M_k) w_k(M_\ell) = \sum_{k=0}^{G-1} v(M_k) \delta_{k,\ell} = v(M_\ell).$$

Ainsi, v et v_h coïncident en tous les nœuds du maillage. Comme ces fonctions sont affines par morceaux sur chaque triangle, elles sont entièrement déterminées par leurs valeurs aux sommets. On en déduit :

$$v = v_h.$$

Donc (w_k) engendre V_h .

2. Indépendance linéaire.

Supposons que

$$\sum_{k=0}^{G-1} \alpha_k w_k = 0.$$

En évaluant en M_ℓ , on obtient :

$$\sum_{k=0}^{G-1} \alpha_k w_k(M_\ell) = \sum_{k=0}^{G-1} \alpha_k \delta_{k,\ell} = \alpha_\ell = 0.$$

Donc tous les coefficients sont nuls, ce qui montre que la famille est libre.

Conclusion.

La famille $(w_k)_{0 \leq k \leq G-1}$ est libre et génératrice de V_h . C'est donc une base de V_h .

28 (f). Soit $k \leq G - 1$ fixé et $T \in \mathcal{T}$ un triangle.

(i) Cas où M_k n'est pas un sommet de T .

Par définition, $w_k(M_\ell) = \delta_{k,\ell}$ pour tout ℓ . Ainsi, si M_k n'est pas un sommet de T , alors w_k est nul aux trois sommets de T .

Comme w_k est affine sur T , elle est entièrement déterminée par ses valeurs aux sommets. On en déduit :

$$w_k|_T = 0.$$

(ii) Cas où M_k est un sommet de T .

Dans ce cas, w_k vaut 1 en M_k et 0 aux deux autres sommets de T .

Par définition des coordonnées barycentriques $(\lambda_0, \lambda_1, \lambda_2)$ associées au triangle T , la fonction affine qui vaut 1 en un sommet et 0 aux deux autres est précisément la coordonnée barycentrique correspondante.

Ainsi, si M_k correspond au sommet A_ℓ de T , alors :

$$w_k|_T = \lambda_\ell.$$

(iii) Expression de $\nabla w_k \cdot \nabla w_j$.

On suppose que M_k et M_j sont deux sommets de T . Soit $F_T : \hat{T} \rightarrow T$ une transformation affine de matrice B_T telle que :

$$F_T^{-1}(M_k) = A_\ell, \quad F_T^{-1}(M_j) = A_s,$$

avec $0 \leq \ell \leq 2$ et $0 \leq s \leq 2$.

D'après la question précédente :

$$w_k|_T = \lambda_\ell, \quad w_j|_T = \lambda_s.$$

On utilise la transformation des gradients par changement de variable affine :

$$\nabla \lambda_\ell^{(T)} = B_T^{-T} \nabla \hat{\lambda}_\ell, \quad \nabla \lambda_s^{(T)} = B_T^{-T} \nabla \hat{\lambda}_s,$$

où $\hat{\lambda}_\ell$ et $\hat{\lambda}_s$ sont les coordonnées barycentriques sur le triangle de référence.

Ainsi,

$$\nabla w_k \cdot \nabla w_j = \nabla \lambda_\ell^{(T)} \cdot \nabla \lambda_s^{(T)} = (B_T^{-T} \nabla \hat{\lambda}_\ell) \cdot (B_T^{-T} \nabla \hat{\lambda}_s).$$

On obtient donc :

$$\nabla w_k \cdot \nabla w_j = \nabla \hat{\lambda}_\ell^T (B_T^{-1} B_T^{-T}) \nabla \hat{\lambda}_s.$$

28 (g). On considère le problème approché : trouver $u_h \in V_h$ vérifiant

$$\forall v_h \in V_h, \quad a(u_h, v_h) = \ell(v_h).$$

Comme $(w_k)_{0 \leq k \leq G-1}$ est une base de V_h , on peut écrire :

$$u_h = \sum_{j=0}^{G-1} U_j w_j,$$

où $(U_j)_{0 \leq j \leq G-1}$ sont des coefficients réels inconnus.

Soit maintenant $v_h \in V_h$. En particulier, on peut choisir $v_h = w_k$ pour $0 \leq k \leq G-1$. On obtient :

$$a(u_h, w_k) = \ell(w_k).$$

En remplaçant u_h par son expression, on a :

$$a \left(\sum_{j=0}^{G-1} U_j w_j, w_k \right) = \ell(w_k).$$

Par linéarité de a par rapport à la première variable :

$$\sum_{j=0}^{G-1} U_j a(w_j, w_k) = \ell(w_k).$$

On obtient donc, pour tout $0 \leq k \leq G-1$:

$$\sum_{j=0}^{G-1} a(w_j, w_k) U_j = \ell(w_k).$$

Ce système peut s'écrire sous forme matricielle :

$$AU = B,$$

où :

$$A = (a_{k,j})_{0 \leq k, j \leq G-1}, \quad a_{k,j} = a(w_j, w_k),$$

$$U = (U_0, \dots, U_{G-1})^T, \quad B = (b_0, \dots, b_{G-1})^T, \quad b_k = \ell(w_k).$$

Conclusion.

Le problème variationnel discret est équivalent à la résolution d'un système linéaire de taille $G \times G$ de la forme $AU = B$.

28 (h) Montrons que la matrice A est inversible et que le problème approché admet une unique solution.

Soit $V = (V_0, \dots, V_{G-1})^T \in R^G$ un vecteur quelconque et considérons la fonction

$$v_h = \sum_{k=0}^{G-1} V_k w_k \in V_h.$$

On a alors :

$$V^T AV = \sum_{k=0}^{G-1} \sum_{j=0}^{G-1} V_k a_{k,j} V_j = \sum_{k=0}^{G-1} \sum_{j=0}^{G-1} V_k a(w_j, w_k) V_j.$$

Par bilinéarité de $a(\cdot, \cdot)$, on obtient :

$$V^T AV = a \left(\sum_{j=0}^{G-1} V_j w_j, \sum_{k=0}^{G-1} V_k w_k \right) = a(v_h, v_h).$$

D'après la définition de la forme bilinéaire $a(\cdot, \cdot)$ (cas elliptique), on a :

$$a(v_h, v_h) = \int_{\Omega} \eta(x, y) |\nabla v_h|^2 dx dy + \int_{\Omega} \mu(x, y) v_h^2 dx dy.$$

Sous les hypothèses usuelles (par exemple $\eta(x, y) \geq \eta_0 > 0$ et $\mu(x, y) \geq 0$), on en déduit que :

$$a(v_h, v_h) \geq 0,$$

et

$$a(v_h, v_h) = 0 \implies v_h = 0.$$

Ainsi,

$$V^T A V = 0 \implies V = 0,$$

ce qui montre que la matrice A est définie positive.

Par conséquent, A est inversible.

On en déduit que le système linéaire

$$AU = B$$

admet une unique solution U , et donc le problème approché admet une unique solution $u_h \in V_h$.

28 (i)

Fonction `matvec` ayant comme paramètre d'entrée un vecteur V quelconque de taille G (ainsi que d'autres paramètres d'entrée nécessaires pour le calcul) et renvoyant à la sortie le vecteur $W = AV$.

```
// Produit matrice-vecteur global A*V
// W[k] = sum_T sum_r V[j] * (diff[i][r] + lambda*reac[i][r])
vector<double> matvec(vector<double> V, int N, int M, double
    a, double b, double lambda, double (*eta_coef)(double,
    double))
{
    // table de connectivite
    vector<vector<int>> TRG = maillageTR(N, M);
    // nombre de noeuds et de triangles
    int G = (N+1)*(M+1);
    int K = 2*M*N;
    // subdivisions en x et y
    vector<double> sub_a = Subdiv(a, N);
    vector<double> sub_b = Subdiv(b, M);
    // coordonnees de tous les noeuds
    vector<double> xs(G), ys(G);
    for (int j = 0; j <= M; j++)
        for (int i = 0; i <= N; i++) {
            int s = numgb(N, M, i, j);
            xs[s] = sub_a[i];
```

```

        ys[s] = sub_b[j];
    }
    // integrales de eta sur chaque triangle
    vector<double> ET = integ_eta_triangu(eta_coef, TRG, xs,
        ys);
    // vecteur resultat initialise a zero
    vector<double> W(G, 0.0);
    // matrices locales et coordonnees
    vector<vector<double>> coord(2, vector<double>(3));
    vector<vector<double>> diff(3, vector<double>(3));
    vector<vector<double>> reac(3, vector<double>(3));
    int k, j;
    double res, PROD2;
    // boucle sur les triangles
    for (int t = 0; t < K; t++) {
        // indices globaux des 3 sommets
        int n0 = TRG[t][0], n1 = TRG[t][1], n2 = TRG[t][2];
        // coordonnees des sommets
        coord[0] = {xs[n0], xs[n1], xs[n2]};
        coord[1] = {ys[n0], ys[n1], ys[n2]};
        // matrices locales de diffusion et reaction
        diff = DiffTerm(coord[0], coord[1], ET[t]);
        reac = ReacTerm(coord[0], coord[1]);
        // boucle sur les sommets locaux i vers noeud global
        k
        for (int i = 0; i < 3; i++) {
            // numero global du sommet i
            k = TRG[t][i];
            res = 0.0;
            // boucle sur les sommets locaux r vers noeud
            global j
            for (int r = 0; r < 3; r++) {
                // numero global du sommet r
                j = TRG[t][r];
                // PROD2 = a^T(w_j, w_k) = diff + lambda*reac
                PROD2 = diff[i][r] + lambda * reac[i][r];
                res = res + V[j] * PROD2;
            } // fin boucle r
            // accumulation W[k] += res
            W[k] = W[k] + res;
        } // fin boucle i
    } // fin boucle t
    return W;
}

```

28 (j)

Fonction `scdmembre` ayant comme paramètre d'entrée une fonction `rhsf` représentant f (ainsi que d'autres paramètres d'entrée nécessaires pour le calcul), et renvoyant à la sortie le vecteur B (ou plutôt une approximation de ce

vecteurs puisque les formules de quadrature ne sont pas toujours exactes). On pourra adopter ici aussi un balayage sur les triangles (et non sur les noeuds) comme dans le produit matrice-vecteur. Ainsi, pour chaque triangle T , on calcule sa contribution aux trois termes b_k correspondant à ses sommets.

```
// Second membre f(x,y) = cos(m*pi*x)*cos(m*pi*y)
double rhsf_global = 1; // parametre m global
// Calcule le vecteur second membre B de taille G
// B[s] = sum_T |det(BT)| * sum_t w[t]*f(F_T(v[t]))*
//      lambda_hat_j(v[t])
vector<double> scdmembre(double (*rhsf)(double, double), int
    N,int M, double a, double b)
{
    // table de connectivite
    vector<vector<int>> TRG = maillageTR(N, M);
    // G = nombre total de noeuds
    int G = (N+1)*(M+1);
    // K = nombre total de triangles
    int K = 2*N*M;
    //On initialise B de taille G avec que des zeros
    vector<double> B(G, 0.0);
    // subdivisions en x et y
    vector<double> sub_a = Subdiv(a, N);
    vector<double> sub_b = Subdiv(b, M);
    // coordonnees de tous les noeuds
    vector<double> xs(G), ys(G);
    for (int j = 0; j <= M; j++)
        for (int i = 0; i <= N; i++) {
            int s = numgb(N, M, i, j);
            xs[s] = sub_a[i]; // coordonnee x du noeud s
            ys[s] = sub_b[j]; // coordonnee y du noeud s
        }
    // points de quadrature sur T_hat (points milieux des
    // cotes)
    vector<vector<double>> v =
        {{0.5,0.0},{0.0,0.5},{0.5,0.5}};
    // poids de quadrature (tous egaux a 1/6)
    vector<double> w = {1.0/6, 1.0/6, 1.0/6};
    // coordonnees locales du triangle courant
    vector<vector<double>> coord(2, vector<double>(3));
    // matrice B_T du triangle courant
    vector<vector<double>> BT;
    // indice global du sommet courant
    int s;
    // contribution du triangle au noeud, valeur de f, valeur
    // de lambda_hat_j
    double contrib, f_val, lambda_j;
    // boucle sur tous les triangles
    for (int i = 0; i < K; i++) {
```

```

// indices globaux des 3 sommets du triangle i
int n0 = TRG[i][0], n1 = TRG[i][1], n2 = TRG[i][2];
// coordonnees x des 3 sommets
coord[0] = {xs[n0], xs[n1], xs[n2]};
// coordonnees y des 3 sommets
coord[1] = {ys[n0], ys[n1], ys[n2]};
// calcul de B_T pour ce triangle
BT = CalcMatBT(coord[0], coord[1]);
// |det(B_T)| = 2 * aire du triangle
double det = abs(BT[0][0]*BT[1][1] - BT[0][1]*BT
[1][0]);
// boucle sur les 3 sommets du triangle
for (int j = 0; j < 3; j++) {
    // numero global du sommet j
    s = TRG[i][j];
    // initialisation de la contribution de ce
    triangle a B[s]
    contrib = 0.0;
    // boucle sur les 3 points de quadrature
    for (int t = 0; t < 3; t++) {
        // transformation du point de quadrature :
        F_T(v[t])
        double xq = coord[0][0] + BT[0][0]*v[t][0] +
            BT[0][1]*v[t][1];
        double yq = coord[1][0] + BT[1][0]*v[t][0] +
            BT[1][1]*v[t][1];
        // evaluation de f au point transforme
        f_val = rhsf(xq, yq);
        // evaluation de lambda_hat_j au point de
        quadrature v[t]
        // depend du sommet local j (pas du point de
        quadrature t)
        if (j == 0) lambda_j = 1.0 - v[t][0] - v[t
][1]; // lambda_hat_0
        else if (j == 1) lambda_j = v[t][0];
                                // lambda_hat_1
        else                    lambda_j = v[t][1];
                                // lambda_hat_2
        // accumulation : poids * f * lambda_hat_j
        contrib += w[t] * f_val * lambda_j;
    }
    // multiplication par |det(B_T)| et accumulation
    dans B[s]
    B[s] += det * contrib;
}
}
return B;
}

```

29 (k) Montrer que

$$V^T AV = a(v, v) = \|\sqrt{\eta} \nabla v\|_{L^2(\Omega)^2}^2 + \lambda \|v\|_{L^2(\Omega)}^2.$$

Soit $v = \sum_{k=0}^{G-1} v_k w_k \in V_h$ et $V = (v_0, \dots, v_{G-1})^T \in R^G$.

On rappelle que la matrice $A = (a_{k,j})$ est définie par

$$a_{k,j} = a(w_j, w_k),$$

où $a(\cdot, \cdot)$ est la forme bilinéaire du problème.

On a alors :

$$V^T AV = \sum_{k=0}^{G-1} \sum_{j=0}^{G-1} v_k a_{k,j} v_j = \sum_{k=0}^{G-1} \sum_{j=0}^{G-1} v_k a(w_j, w_k) v_j.$$

Par bilinéarité de $a(\cdot, \cdot)$, on obtient :

$$V^T AV = a\left(\sum_{j=0}^{G-1} v_j w_j, \sum_{k=0}^{G-1} v_k w_k\right) = a(v, v).$$

D'après la définition de la forme bilinéaire, on a :

$$a(v, v) = \int_{\Omega} \eta(x, y) |\nabla v(x, y)|^2 dx dy + \lambda \int_{\Omega} v(x, y)^2 dx dy.$$

On en déduit :

$$V^T AV = \|\sqrt{\eta} \nabla v\|_{L^2(\Omega)^2}^2 + \lambda \|v\|_{L^2(\Omega)}^2.$$

En particulier :

— Si $\eta = 0$ et $\lambda = 1$, alors

$$V^T AV = \|v\|_{L^2(\Omega)}^2.$$

— Si $\eta = 1$ et $\lambda = 0$, alors

$$V^T AV = \|\nabla v\|_{L^2(\Omega)^2}^2.$$

29 (l) Fonction `normL2` ayant comme paramètre d'entrée principal un tableau V de taille G et renvoyant en sortie la norme $\|v\|_{L^2(\Omega)}$.

```

// Norme L2 de v : sqrt(V^t * A * V) avec eta=0, lambda=1
// On utilise seulement ReactTerm (pas de DiffTerm)
double normL2(vector<double> V, int N, int M, double a,
double b) {
    // table de connectivite
    vector<vector<int>> TRG = maillageTR(N, M);
    // nombre de noeuds et de triangles
    int G = (N+1)*(M+1);
    int K = 2*M*N;
    // subdivisions
    vector<double> sub_a = Subdiv(a, N);
    vector<double> sub_b = Subdiv(b, M);
    // coordonnees de tous les noeuds
    vector<double> xs(G), ys(G);
    for (int jj = 0; jj <= M; jj++)
        for (int ii = 0; ii <= N; ii++) {
            int s = numgb(N, M, ii, jj);
            xs[s] = sub_a[ii];
            ys[s] = sub_b[jj];
        }
    // vecteur resultat W = A*V initialise a zero
    vector<double> W(G, 0.0);
    // coordonnees locales et matrice de reaction
    vector<vector<double>> coord(2, vector<double>(3));
    vector<vector<double>> reac(3, vector<double>(3));
    int k, j;
    double res, PROD2;
    // boucle sur les triangles (analogue a matvec avec eta
    =0, lambda=1)
    for (int t = 0; t < K; t++) {
        // indices globaux des 3 sommets
        int n0 = TRG[t][0], n1 = TRG[t][1], n2 = TRG[t][2];
        // coordonnees des sommets
        coord[0] = {xs[n0], xs[n1], xs[n2]};
        coord[1] = {ys[n0], ys[n1], ys[n2]};
        // seulement ReactTerm : eta=0 donc DiffTerm supprime
        reac = ReactTerm(coord[0], coord[1]);
        // boucle sur les sommets locaux i
        for (int i = 0; i < 3; i++) {
            // numero global du sommet i
            k = TRG[t][i];
            res = 0.0;
            // boucle sur les sommets locaux r
            for (int r = 0; r < 3; r++) {
                // numero global du sommet r
                j = TRG[t][r];
                // PROD2 = reac[i][r] car lambda=1 et pas de
                diffusion
            }
        }
    }
}

```

```

        PROD2 = reac[i][r];
        res = res + V[j] * PROD2;
    } // fin boucle r
    // accumulation dans W[k]
    W[k] = W[k] + res;
} // fin boucle i
} // fin boucle t
// ||v||^2_L2 = V^t * W, on retourne la racine
double norme2 = 0.0;
for (int k = 0; k < G; k++)
    norme2 += V[k] * W[k];

return sqrt(norme2);
}

```

29 (m) Fonction normL2Grad ayant comme paramètre d'entrée principal un tableau V de taille I et renvoyant en sortie la norme $\|\nabla v\|_{L^2(\Omega)}$.

```

// Norme H1 semi-normee de v : sqrt(V^t * A * V) avec eta=1,
// lambda=0
// On utilise seulement DiffTerm (pas de ReacTerm)
double normL2Grad(vector<double> V, int N, int M, double a,
double b) {
    // table de connectivite
    vector<vector<int>> TRG = maillageTR(N, M);
    // nombre de noeuds et de triangles
    int G = (N+1)*(M+1);
    int K = 2*M*N;
    // subdivisions
    vector<double> sub_a = Subdiv(a, N);
    vector<double> sub_b = Subdiv(b, M);
    // coordonnees de tous les noeuds
    vector<double> xs(G), ys(G);
    for (int jj = 0; jj <= M; jj++)
        for (int ii = 0; ii <= N; ii++) {
            int s = numgb(N, M, ii, jj);
            xs[s] = sub_a[ii];
            ys[s] = sub_b[jj];
        }
    // ET[t] = |det(BT)|/2 car eta=1 donc int_T eta = aire(T)
    vector<double> ET(K);
    for (int t = 0; t < K; t++) {
        int n0=TRG[t][0], n1=TRG[t][1], n2=TRG[t][2];
        double x0=xs[n0],y0=ys[n0];
        double x1=xs[n1],y1=ys[n1];
        double x2=xs[n2],y2=ys[n2];
        // |det(BT)| = 2*aire => int_T 1 = |det|/2
        ET[t] = abs((x1-x0)*(y2-y0)-(x2-x0)*(y1-y0)) / 2.0;
    }
}

```



```

// vecteur resultat W = A*V initialise a zero
vector<double> W(G, 0.0);
// coordonnees locales et matrice de diffusion
vector<vector<double>> coord(2, vector<double>(3));
vector<vector<double>> diff(3, vector<double>(3));
int k, j;
double res, PROD2;
// boucle sur les triangles (analogue a matvec avec eta
    =1, lambda=0)
for (int t = 0; t < K; t++) {
    // indices globaux des 3 sommets
    int n0 = TRG[t][0], n1 = TRG[t][1], n2 = TRG[t][2];
    // coordonnees des sommets
    coord[0] = {xs[n0], xs[n1], xs[n2]};
    coord[1] = {ys[n0], ys[n1], ys[n2]};
    // seulement DiffTerm avec ET[t] = aire(T) car eta=1
    diff = DiffTerm(coord[0], coord[1], ET[t]);
    // boucle sur les sommets locaux i
    for (int i = 0; i < 3; i++) {
        // numero global du sommet i
        k = TRG[t][i];
        res = 0.0;
        // boucle sur les sommets locaux r
        for (int r = 0; r < 3; r++) {
            // numero global du sommet r
            j = TRG[t][r];
            // PROD2 = diff[i][r] car lambda=0 (pas de
                ReacTerm)
            PROD2 = diff[i][r];
            res = res + V[j] * PROD2;
        } // fin boucle r
        // accumulation dans W[k]
        W[k] = W[k] + res;
    } // fin boucle i
} // fin boucle t
// ||grad v||^2_L2 = V^t * W, on retourne la racine
double norme2 = 0.0;
for (int k = 0; k < G; k++)
    norme2 += V[k] * W[k];

return sqrt(norme2);
}

```

4 Inversion du système : Méthode du Gradient Biconjugué Stabilisé (BICGSTAB)

Programme qui utilise la méthode de gradient biconjugué stabilisé pour résoudre le système $AX = B$ ci-dessus (et qui utilise la fonction produit matrice-vecteur ci-dessus).

```
// methode du gradient biconjugué stabilisé BICGSTAB pour
résoudre  $A \cdot X = B$ 
vector<double> bicgstab(
    vector<double> B,
    int N, int M,
    double a, double b,
    double lambda,
    double (*eta_coef)(double, double))
{
    // tolerance de convergence
    double tol= 1e-6;
    // nombre max d'iterations
    int itermax = 1000;
    // taille du systeme
    int n = B.size();
    // X0 = vecteur nul (solution initiale)
    vector<double> X(n, 0.0);
    // R0 = B - A*X0 = B (car X0 = 0)
    vector<double> R = B;
    // R0* = R0 (choix arbitraire)
    vector<double> Rx = R;
    // W0 = R0
    vector<double> W = R;
    // vecteurs intermediaires de l'algorithme
    vector<double> AW(n), Sj(n), ASj(n), Rj(n);
    // scalaires de l'algorithme
    double alpha, omega, beta;
    // convergence initiale = norme du residu initial
    double conv = norme_vect(R);
    // compteur d'iterations
    int j = 0;
    // boucle jusqu'a convergence ou nombre max d'iterations
    while (j < itermax && conv > tol) {
        // AW_j = A * W_j
        AW = matvec(W, N, M, a, b, lambda, eta_coef);
        // alpha_j = <R_j, R0*> / <AW_j, R0*>
        alpha = pdt_scal(R, Rx) / pdt_scal(AW, Rx);
        // S_j = R_j - alpha_j * AW_j
        for (int i =0; i < n; i++)
            Sj[i] = R[i] - alpha * AW[i];
```

```

        // AS_j = A * S_j
ASj = matvec(Sj, N, M, a, b, lambda, eta_coef);
// omega_j = <AS_j, S_j> / <AS_j, AS_j>
omega = pdt_scal(ASj, Sj) / pdt_scal(ASj, ASj);
// X_{j+1} = X_j + alpha_j * W_j + omega_j * S_j
for (int i = 0; i < n; i++)
    X[i] = X[i] + alpha * W[i] + omega * Sj[i];
// R_{j+1} = S_j - omega_j * AS_j
for (int i = 0; i < n; i++)
    Rj[i] = Sj[i] - omega * ASj[i];
// beta_j = (<R_{j+1}, R0*> / <R_j, R0*>) * (alpha_j /
    omega_j)
beta = (pdt_scal(Rj, Rx) / pdt_scal(R, Rx)) * (alpha
    / omega);
// W_{j+1} = R_{j+1} + beta_j * (W_j - omega_j * AW_j
    )
for (int i = 0; i < n; i++)
    W[i] = Rj[i] + beta * (W[i] - omega * AW[i]);
// mise a jour du residu et du compteur
R = Rj;
conv = norme_vect(R);
j++;
}
return X;
}

```

5 Tests Numériques

5.1 Visualisation du maillage

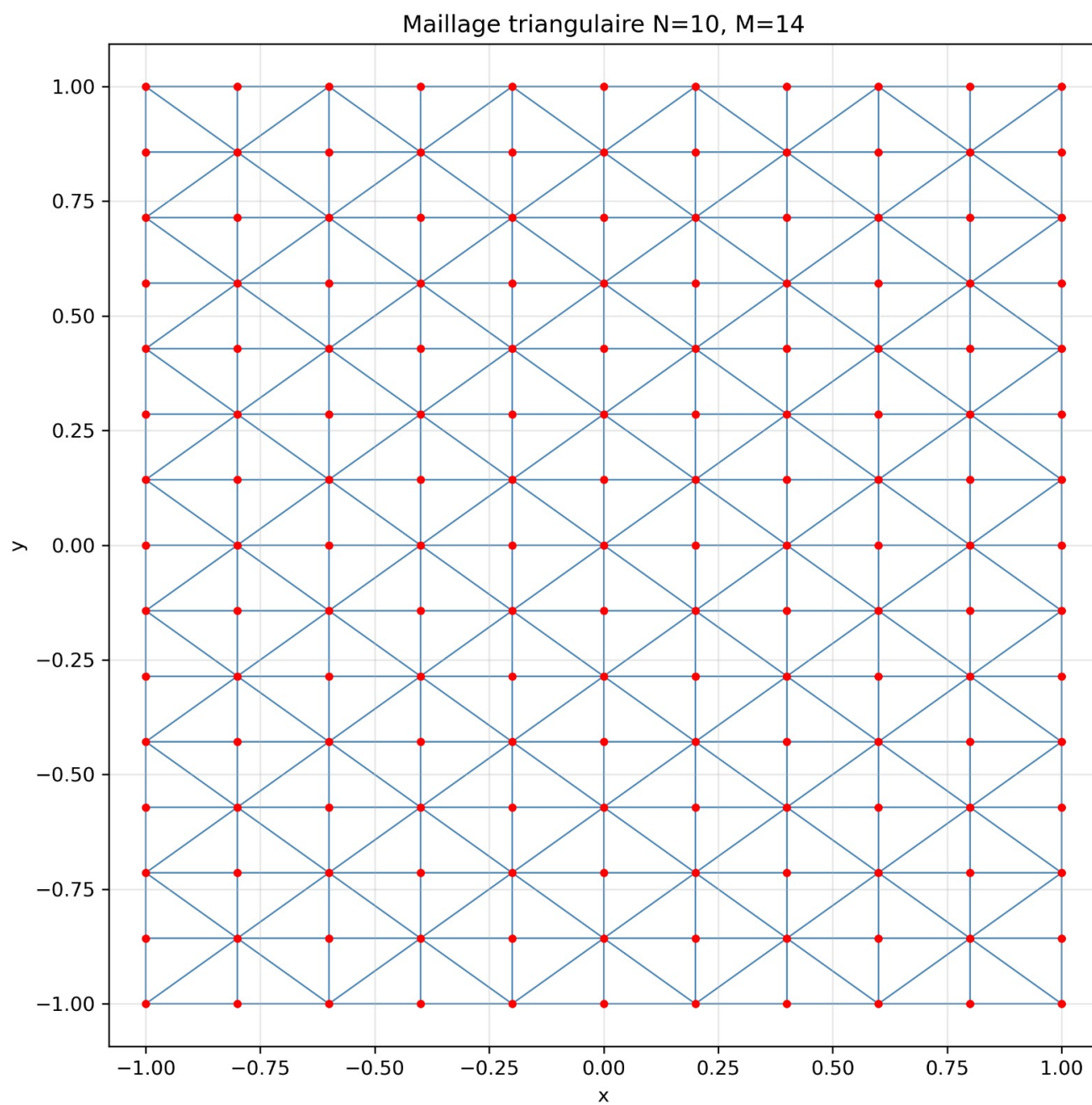


FIGURE 1 – Maillage du domaine pour N=10 et M=14

5.2 Question 31

Nous devons montrer que pour $vh \in V_h$ et $(x, y) \in \Omega$, les indices $i = \lfloor \frac{x+a}{h_1} \rfloor$, $j = \lfloor \frac{y+b}{h_2} \rfloor$ satisfont :

- $x_i \leq x < x_{i+1}$ avec $x_i = ih_1 - a$ et $x_{i+1} = (i+1)h_1 - a$
- $y_j \leq y < y_{j+1}$ avec $y_j = jh_2 - b$ et $y_{j+1} = (j+1)h_2 - b$

Démonstration :

1. Pour i :

$0 \leq \frac{x+a}{h_1} < N$ (car $\frac{x+a}{h_1}$ divise le côté $x+a$ en N sous-intervalles de longueur h_1)

$$0 \leq i < N - 1 \text{ (car } i \text{ est la partie entière de } \frac{x+a}{h_1} \text{)}$$

2. Pour j :

$0 \leq \frac{y+b}{h_2} < M$ (car $\frac{y+b}{h_2}$ divise le côté $y+b$ en M sous-intervalles de longueur h_2)

$$0 \leq j < M - 1 \text{ (car } j \text{ est la partie entière de } \frac{y+b}{h_2} \text{)}$$

3. Pour x dans l'intervalle $[x_i, x_{i+1})$ et y dans l'intervalle $[y_j, y_{j+1})$:

- Montrons $x_i \leq x < x_{i+1}$:

$$\begin{aligned} ih_1 - a &\leq x < (i+1)h_1 - a \\ \Rightarrow ih_1 &\leq x + a < (i+1)h_1 \\ \Rightarrow i &\leq \frac{x+a}{h_1} < i+1 \\ \Rightarrow i &= \left\lfloor \frac{x+a}{h_1} \right\rfloor \end{aligned}$$

- Montrons $y_j \leq y < y_{j+1}$:

$$\begin{aligned} jh_2 - b &\leq y < (j+1)h_2 - b \\ \Rightarrow jh_2 &\leq y + b < (j+1)h_2 \\ \Rightarrow j &\leq \frac{y+b}{h_2} < j+1 \\ \Rightarrow j &= \left\lfloor \frac{y+b}{h_2} \right\rfloor \end{aligned}$$

Dédution :

En résumé, nous avons montré que pour $vh \in V_h$ et $(x, y) \in \Omega$, les indices i et j sont définis tels que x et y appartiennent aux intervalles correspondants. Puisque $x_i \leq x < x_{i+1}$ et $y_j \leq y < y_{j+1}$, cela signifie que (x, y) est situé dans le triangle T_l où T_{l+1} avec $l = 2jN + 2i$.

Cette démonstration justifie la déduction suivante :

$$(x, y) \in T_l \quad \text{ou} \quad (x, y) \in T_{l+1} \quad \text{avec} \quad l = 2jN + 2i$$

5.3 Question 32 (a) — Calcul du second membre f pour $\eta_0 = 0$

On considère l'EDP :

$$-\frac{\partial}{\partial x} \left(\eta \frac{\partial u^p}{\partial x} \right) - \frac{\partial}{\partial y} \left(\eta \frac{\partial u^p}{\partial y} \right) + \lambda u^p = f$$

avec $\eta_0 = 0$, donc $\eta(x, y) = 1$, $\lambda = 1$ et la solution particulière :

$$u^p(x, y) = \frac{\cos(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1}.$$

Calcul des dérivées partielles

Dérivée en x :

$$\frac{\partial u^p}{\partial x} = \frac{-m\pi \sin(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1}$$

$$-\frac{\partial}{\partial x} \left(\frac{\partial u^p}{\partial x} \right) = \frac{m^2\pi^2 \cos(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1}$$

Dérivée en y :

$$\frac{\partial u^p}{\partial y} = \frac{-m\pi \cos(m\pi x) \sin(m\pi y)}{2m^2\pi^2 + 1}$$

$$-\frac{\partial}{\partial y} \left(\frac{\partial u^p}{\partial y} \right) = \frac{m^2\pi^2 \cos(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1}$$

Terme de réaction :

$$\lambda u^p = \frac{\cos(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1}$$

Calcul de f

En additionnant les trois termes :

$$\begin{aligned} f(x, y) &= -\frac{\partial}{\partial x} \left(\frac{\partial u^p}{\partial x} \right) - \frac{\partial}{\partial y} \left(\frac{\partial u^p}{\partial y} \right) + \lambda u^p \\ &= \frac{m^2\pi^2 \cos(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1} + \frac{m^2\pi^2 \cos(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1} + \frac{\cos(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1} \\ &= \frac{(2m^2\pi^2 + 1) \cos(m\pi x) \cos(m\pi y)}{2m^2\pi^2 + 1} \\ &= \cos(m\pi x) \cos(m\pi y). \end{aligned}$$

5.4 Question 32 (b)

- Fonction `erreurs` ayant comme paramètres d'entrée la fonction `solExa` (qui renvoie la valeur de la solution exacte en tout point (x, y)), la solution approchée u_h et qui renvoie les trois erreurs relatives :

$$e_0(h) = \frac{\|\hat{u}_h - u_h\|_{L^2(\Omega)}}{\|\hat{u}_h\|_{L^2(\Omega)}}, \quad e_1(h) = \frac{\|\nabla \hat{u}_h - \nabla u_h\|_{L^2(\Omega)^2}}{\|\nabla \hat{u}_h\|_{L^2(\Omega)^2}}, \quad e_\infty(h) = \frac{\|\hat{u}_h - u_h\|_\infty}{\|\hat{u}_h\|_\infty}$$

où $\|\hat{u}_h\|_\infty := \sup_{0 \leq k \leq G-1} |\hat{u}_h(M_k)|$.

```
// Constante pi
const double pi = 3.141592653589793;
// Parametre m global (entier naturel de l'enonce)
int m_global = 1;
// Solution exacte u^p(x,y) = cos(m*pi*x)*cos(m*pi*y)/(2m^2*
    pi^2+1)
double solExa(double x, double y) {
    return cos(m_global * pi * x) * cos(m_global * pi * y)
        / (2.0 * m_global*m_global * pi*pi + 1.0);
}
// Second membre f(x,y) = cos(m*pi*x)*cos(m*pi*y)
double rhsf(double x, double y) {
    return cos(m_global * pi * x) * cos(m_global * pi * y);
}
// Calcule les 3 erreurs relatives entre u^h et u^p
// e0 = ||uh_hat - uh||_L2 / ||uh_hat||_L2
// e1 = ||grad(uh_hat-uh)||_L2 / ||grad(uh_hat)||_L2
// einf = max|uh_hat - uh| / max|uh_hat|
vector<double> erreurs(
    double (*solExa)(double, double),
    vector<double> uh, // solution approchee de taille G
    int N, int M,
    double a, double b)
{
    // nombre total de noeuds
    int G = (N+1)*(M+1);
    // subdivisions
    vector<double> sub_a = Subdiv(a, N);
    vector<double> sub_b = Subdiv(b, M);
    // coordonnees de tous les noeuds
    vector<double> xs(G), ys(G);
    for (int j = 0; j <= M; j++)
        for (int i = 0; i <= N; i++) {
            int s = numgb(N, M, i, j);
            xs[s] = sub_a[i];
            ys[s] = sub_b[j];
        }
}
```

```

// uh_hat = interpolée de u^p aux noeuds du maillage
vector<double> uh_hat(G);
for (int k = 0; k < G; k++)
    uh_hat[k] = solExa(xs[k], ys[k]);
// diff_vec = uh_hat - uh (différence noeud par noeud)
vector<double> diff_vec(G);
for (int k = 0; k < G; k++)
    diff_vec[k] = uh_hat[k] - uh[k];
// e0 = erreur relative en norme L2
double e0 = normL2(diff_vec, N, M, a, b)
           / normL2(uh_hat, N, M, a, b);
// e1 = erreur relative en semi-norme H1 (gradient)
double e1 = normL2Grad(diff_vec, N, M, a, b)
           / normL2Grad(uh_hat, N, M, a, b);
// calcul de l'erreur infinie
double maxdiff = 0.0; // max de |uh_hat - uh|
double maxw     = 0.0; // max de |uh_hat|

for (int k = 0; k < G; k++) {
    // mise à jour du max de l'erreur absolue
    if (abs(diff_vec[k]) > maxdiff)
        maxdiff = abs(diff_vec[k]);
    // mise à jour du max de la solution exacte
    if (abs(uh_hat[k]) > maxw)
        maxw = abs(uh_hat[k]);
}
// einf = erreur relative en norme infinie
double einf = maxdiff / maxw;
// retourne les 3 erreurs
return {e0, e1, einf};
}

```

5.5 Affichage des valeurs numériques des erreurs Pour différents N

N	h	e0	e1	einf
5	4.000e-01	1.312e-01	2.193e-01	2.551e-01
10	2.000e-01	2.588e-02	1.058e-01	6.223e-02
20	1.000e-01	6.031e-03	5.248e-02	1.475e-02
40	5.000e-02	1.486e-03	2.620e-02	3.871e-03
80	2.500e-02	4.764e-04	1.309e-02	1.150e-03

=== Code Execution Successful ===

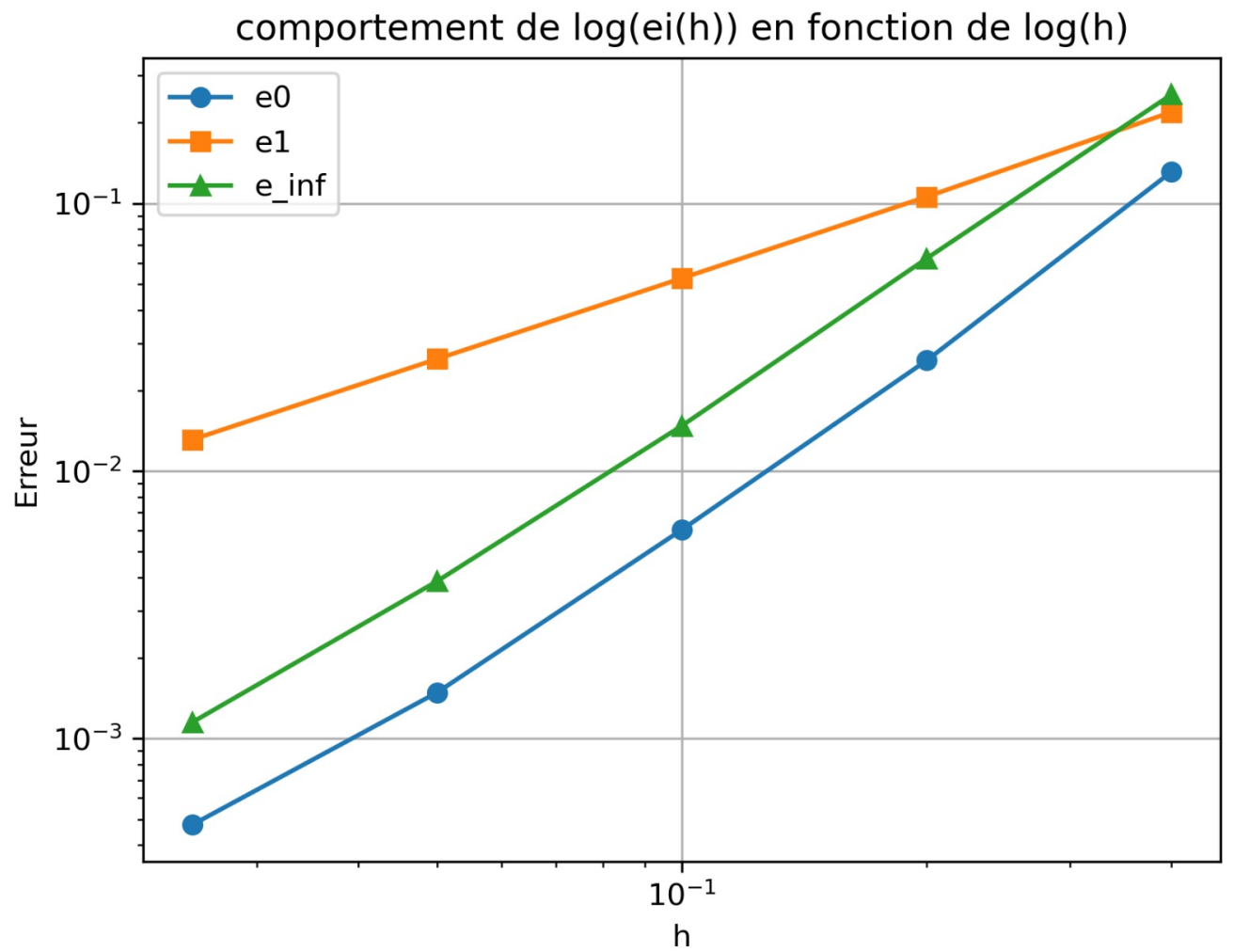


FIGURE 2 – Comportement de $\log(e_i(h))$ en fonction de $\log(h)$

5.6 Question 32 (d) Visualisation des courbes des fonctions pour différentes valeurs de m

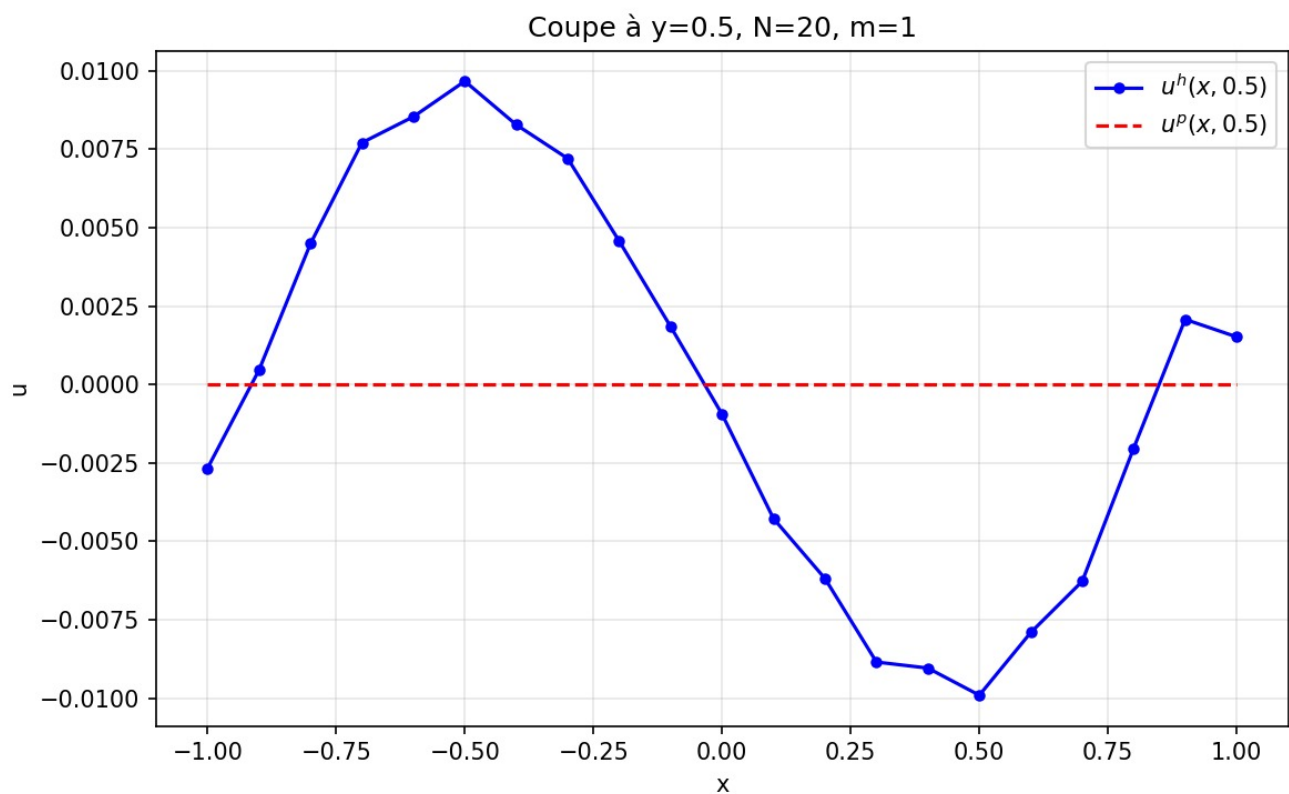


FIGURE 3 – $N=20$ et $M=1$

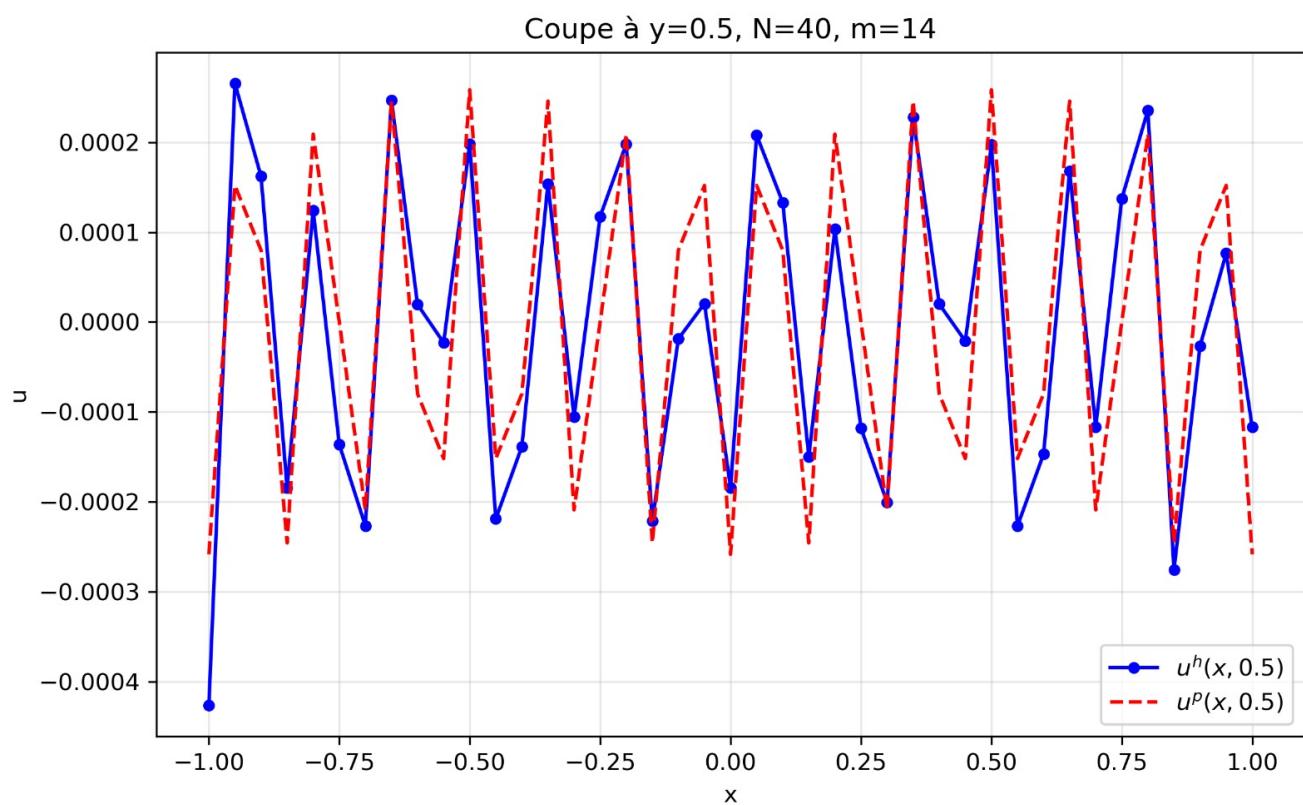


FIGURE 4 – $N=40$ et $M=14$

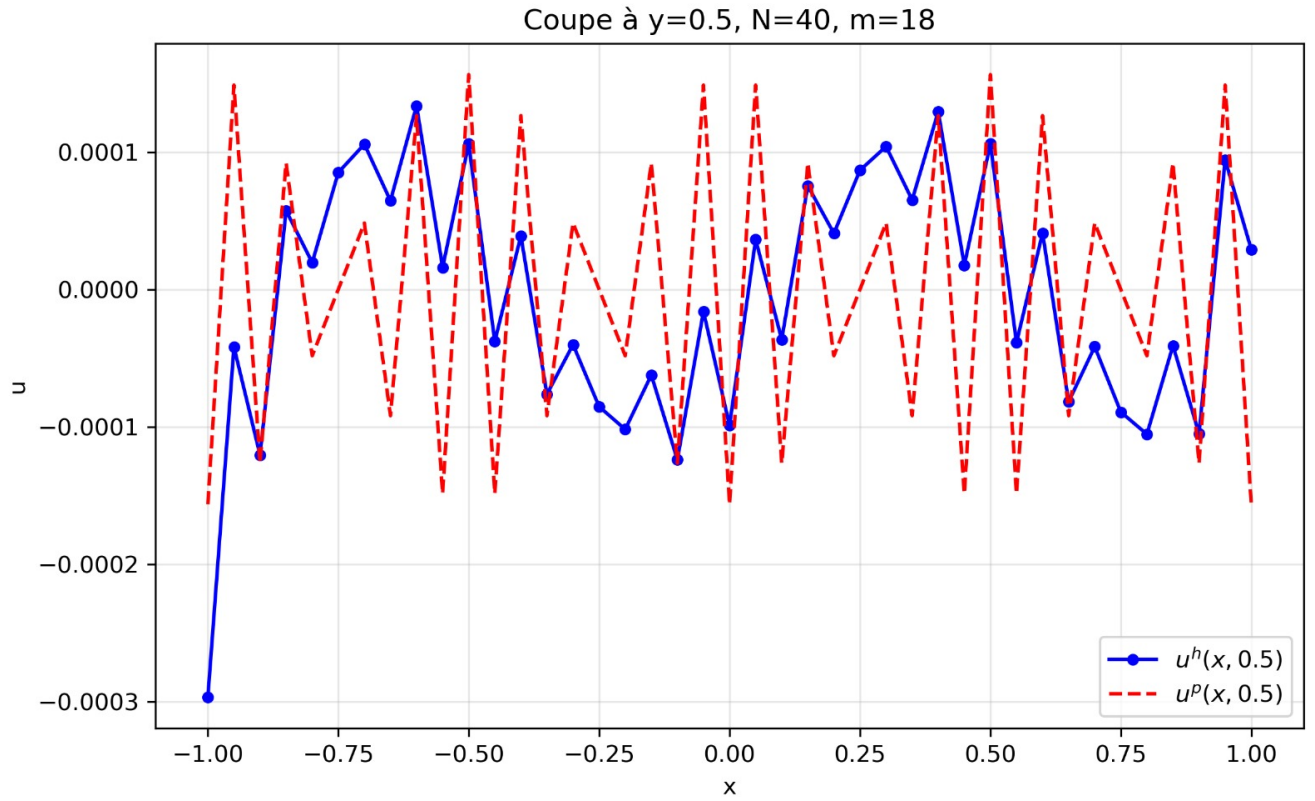


FIGURE 5 – $N=40$ et $M=18$

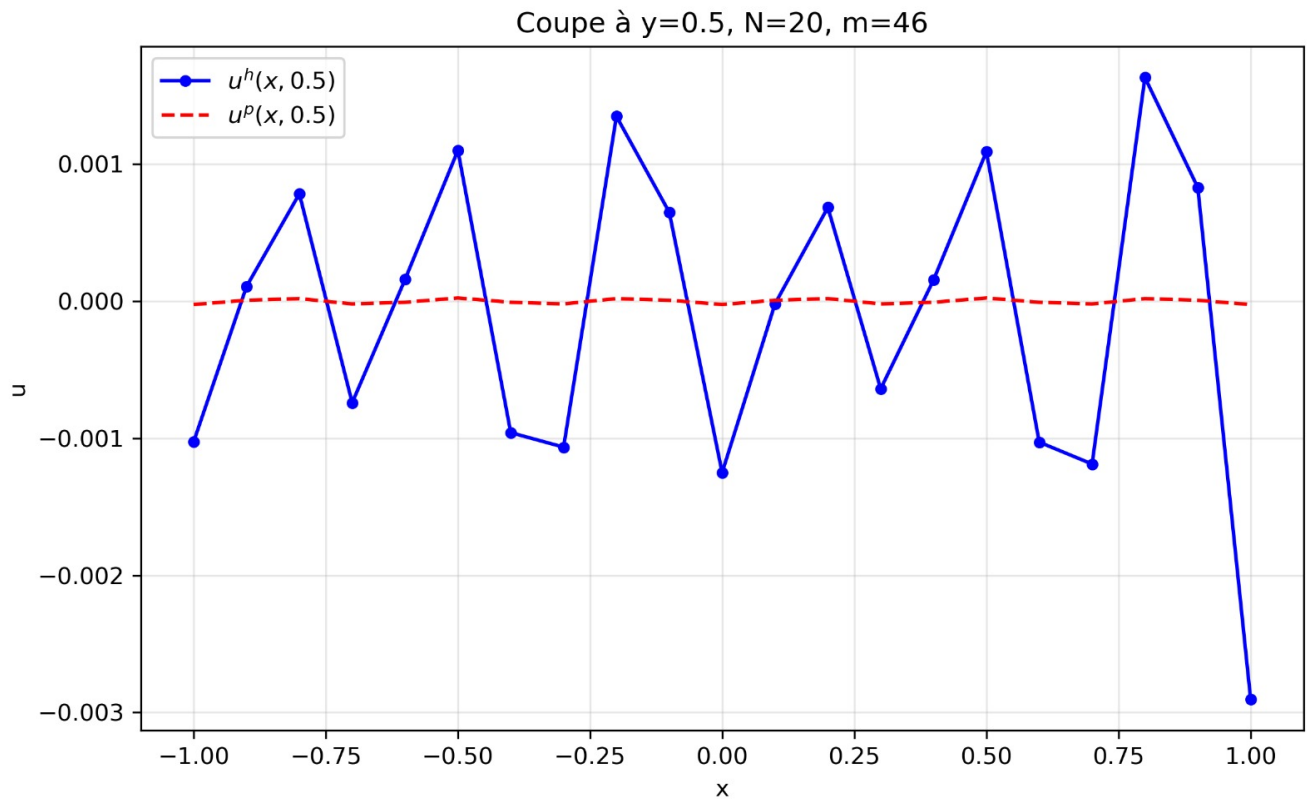


FIGURE 6 – $N=20$ et $M=46$

5.7 Code C++ complet du projet

```
#include <iostream>
#include <vector>
#include <tuple>
#include <cmath>
#include <array>
#include <math.h>
#include <iomanip>
#include <fstream>
using namespace std;
// on vas faire une subdivision uniforme de [-a, a] en N+1
points
vector<double> Subdiv(double a, int N){
    //on cree un vecteur de taille N+1 contenant les points
    de subdivision
    vector<double> xi(N+1);
    for(int i = 0 ; i<=N ; i++){
        // xi[i] = -a + i * pas, avec pas = 2a/N
        xi[i] = -a + i * (2*a) / N;
    }
    return xi;
}
// numero global d'un noeud (i,j) dans la grille
// on applique la formule : s = (N+1)*j + i
int numgb(int N, int M, int i, int j){
    return (N+1)*j + i;
}
// On inverse de numgb : retrouve (i,j) depuis le numero
global S
// i = S mod (N+1) et j = S / (N+1)
vector<int> invnumgb(int N, int M, int S){
    int i = S % (N+1); // reste de la division euclidienne =
    indice en x
    int j = S / (N+1); // quotient de la division euclidienne
    = indice en y
    return {i, j};
}
//On fait la construction du maillage triangulaire
// On vas retourner une matrice TRG de taille K x 3, K = 2*N*M
triangles
// Chaque ligne va contenir les 3 indices globaux des sommets
vector<vector<int>> maillageTR(int N, int M) {
    // K = nombre total de triangles
    int K = 2 * N * M;
    // TRG : table de connectivite
    vector<vector<int>> TRG(K);
    // l : indice du triangle courant
```

```

int l = 0;
// parcours de chaque cellule rectangulaire (i,j)
for (int j = 0; j < M; j++) {
    for (int i = 0; i < N; i++) {
        // indices globaux des 4 coins de la cellule
        int s1 = numgb(N,M,i, j ); // bas-gauche
        int s2 = numgb(N,M,i+1,j ); // bas-droite
        int s3 = numgb(N,M,i, j+1); // haut-gauche
        int s4 = numgb(N,M,i+1,j+1); // haut-droite
        // decoupage en 2 triangles selon la parite de i+j
        if ((i + j) % 2 == 0) {
            // diagonale de s2 vers s3 (bas-droite ->
            // haut-gauche)
            TRG[l++] = {s1, s2, s4}; // triangle T-
            TRG[l++] = {s1, s3, s4}; // triangle T+
        } else {
            // diagonale de s1 vers s4 (bas-gauche ->
            // haut-droite)
            TRG[l++] = {s1, s2, s3}; // triangle T-
            TRG[l++] = {s2, s3, s4}; // triangle T+
        }
    }
}
return TRG;
}

// Calcule la matrice jacobienne B_T du triangle T
vector<vector<double>> CalcMatBT(vector<double> xs, vector<
double> ys) {
    // BT est une matrice 2x2
    vector<vector<double>> BT(2, vector<double>(2));
    // premiere ligne : differences en x
    BT[0] = {xs[1] - xs[0], xs[2] - xs[0]};
    // deuxieme ligne : differences en y
    BT[1] = {ys[1] - ys[0], ys[2] - ys[0]};
    return BT;
}

// Coefficient de diffusion eta(x,y) = 1 - (eta0/4)*(x^2+y^2)
// ici eta0 = 0.5 fixe pour le test
double eta_coef(double x, double y){
    return 1;
}

// Calcule ET[l] = integrale de eta sur le triangle T_l
// par la formule de quadrature des points milieux
// ET[l] = |det(B_T)| * (1/6) * sum eta(points milieux)
vector<double> integ_eta_triangu(
    double (*eta_coef)(double, double),
    const vector<vector<int>>& TRG,
    const vector<double>& xs,
    const vector<double>& ys)

```

```

{
    // K = nombre de triangles
    int K = TRG.size();
    // ET[l] contiendra l'integrale de eta sur le triangle l
    vector<double> ET(K);

    for (int l = 0; l < K; l++) {
        // On recupere les indices globaux des 3 sommets
        int n0 = TRG[l][0], n1 = TRG[l][1], n2 = TRG[l][2];
        // coordonnees des 3 sommets
        double x0 = xs[n0], y0 = ys[n0];
        double x1 = xs[n1], y1 = ys[n1];
        double x2 = xs[n2], y2 = ys[n2];
        // determinant de B_T = 2 * aire du triangle
        double detBT = (x1-x0)*(y2-y0) - (x2-x0)*(y1-y0);
        // 1er point de quadrature : milieu du cote [A0,A1]
        double px1 = x0 + 0.5*(x1-x0);
        double py1 = y0 + 0.5*(y1-y0);
        // 2eme point de quadrature : milieu du cote [A0,A2]
        double px2 = x0 + 0.5*(x2-x0);
        double py2 = y0 + 0.5*(y2-y0);
        // 3eme point de quadrature : milieu du cote [A1,A2]
        double px3 = x0 + 0.5*(x1-x0) + 0.5*(x2-x0);
        double py3 = y0 + 0.5*(y1-y0) + 0.5*(y2-y0);
        // formule des points milieux : poids 1/6 pour chaque
        // point
        double quadrature = (1.0/6.0) * (eta_coef(px1,py1)
                                         + eta_coef(px2,py2)
                                         + eta_coef(px3,py3));
        // changement de variable : multiplication par |det(B_T)|
        ET[l] = abs(detBT) * quadrature;
    }
    return ET;
}

// Calcule la matrice de reaction locale 3x3
// R[i][r] = |det(B_T)| * int_T_hat lambda_hat_i *
//           lambda_hat_r
// = |det(B_T)| * { 1/12 si i==r, 1/24 sinon }
vector<vector<double>> ReacTerm(vector<double> xs, vector<
double> ys) {
    // determinant de B_T
    double det = (xs[1]-xs[0])*(ys[2]-ys[0])
                - (xs[2]-xs[0])*(ys[1]-ys[0]);
    // valeur absolue du determinant
    double absdet = abs(det);
    // matrice de reaction 3x3
    vector<vector<double>> R(3);
    // diagonale : absdet/12, hors-diagonale : absdet/24
    R[0] = {absdet/12.0, absdet/24.0, absdet/24.0};
}

```

```

    R[1] = {absdet/24.0, absdet/12.0, absdet/24.0};
    R[2] = {absdet/24.0, absdet/24.0, absdet/12.0};
    return R;
}
// Calcule la matrice de diffusion locale 3x3
// D[i][r] = val * (grad_lambda_i . grad_lambda_r)
// ou val = ET[t] = integrale de eta sur T
vector<vector<double>> DiffTerm(vector<double> xs, vector<
double> ys, double val) {
    // coefficients de B_T
    double a = xs[1]-xs[0], b = xs[2]-xs[0];
    double c = ys[1]-ys[0], d = ys[2]-ys[0];
    // determinant de B_T
    double det = a*d - b*c;
    // inverse de B_T : B_T^{-1} = (1/det) * | d  -b |
    //                                         | -c  a |
    double inv[2][2] = {{ d/det, -b/det},
                        {-c/det,  a/det}};
    // gradients des fonctions de base sur T_hat
    // grad_hat_lambda_0 = (-1,-1), grad_hat_lambda_1 = (1,0)
    // , grad_hat_lambda_2 = (0,1)
    double gh[3][2] = {{-1,-1},{1,0},{0,1}};
    // gradients sur T via g[i] = B_T^{-T} * grad_hat[i]
    double g[3][2];
    for (int i = 0; i < 3; i++) {
        // composante x du gradient transforme
        g[i][0] = inv[0][0]*gh[i][0] + inv[1][0]*gh[i][1];
        // composante y du gradient transforme
        g[i][1] = inv[0][1]*gh[i][0] + inv[1][1]*gh[i][1];
    }
    // matrice de diffusion 3x3
    vector<vector<double>> D(3, vector<double>(3));
    // D[i][j] = val * produit scalaire des gradients
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            D[i][j] = val * (g[i][0]*g[j][0] + g[i][1]*g[j]
                               ][1]);
    return D;
}
// Produit matrice-vecteur global A*V
// W[k] = sum_T sum_r V[j] * (diff[i][r] + lambda*reac[i][r])
vector<double> matvec(vector<double> V, int N, int M, double
a, double b, double lambda, double (*eta_coef)(double,
double))
{
    // table de connectivite
    vector<vector<int>> TRG = maillageTR(N, M);
    // nombre de noeuds et de triangles
    int G = (N+1)*(M+1);
    int K = 2*M*N;

```



```

// subdivisions en x et y
vector<double> sub_a = Subdiv(a, N);
vector<double> sub_b = Subdiv(b, M);
// coordonnees de tous les noeuds
vector<double> xs(G), ys(G);
for (int j = 0; j <= M; j++)
    for (int i = 0; i <= N; i++) {
        int s = numgb(N, M, i, j);
        xs[s] = sub_a[i];
        ys[s] = sub_b[j];
    }
// integrales de eta sur chaque triangle
vector<double> ET = integ_eta_triangu(eta_coef, TRG, xs,
    ys);
// vecteur resultat initialise a zero
vector<double> W(G, 0.0);
// matrices locales et coordonnees
vector<vector<double>> coord(2, vector<double>(3));
vector<vector<double>> diff(3, vector<double>(3));
vector<vector<double>> reac(3, vector<double>(3));
int k, j;
double res, PROD2;
// boucle sur les triangles
for (int t = 0; t < K; t++) {
    // indices globaux des 3 sommets
    int n0 = TRG[t][0], n1 = TRG[t][1], n2 = TRG[t][2];
    // coordonnees des sommets
    coord[0] = {xs[n0], xs[n1], xs[n2]};
    coord[1] = {ys[n0], ys[n1], ys[n2]};
    // matrices locales de diffusion et reaction
    diff = DiffTerm(coord[0], coord[1], ET[t]);
    reac = ReactTerm(coord[0], coord[1]);
    // boucle sur les sommets locaux i vers noeud global
    k
    for (int i = 0; i < 3; i++) {
        // numero global du sommet i
        k = TRG[t][i];
        res = 0.0;
        // boucle sur les sommets locaux r vers noeud
        global j
        for (int r = 0; r < 3; r++) {
            // numero global du sommet r
            j = TRG[t][r];
            // PROD2 = a^T(w_j, w_k) = diff + lambda*reac
            PROD2 = diff[i][r] + lambda * reac[i][r];
            res = res + V[j] * PROD2;
        } // fin boucle r
        // accumulation W[k] += res
        W[k] = W[k] + res;
    } // fin boucle i

```

```

    } // fin boucle t
    return W;
}
// Second membre  $f(x,y) = \cos(m\pi x) \cos(m\pi y)$ 
double rhsf_global = 1; // parametre m global
// Calcule le vecteur second membre B de taille G
//  $B[s] = \sum_T |\det(BT)| * \sum_t w[t] * f(F_T(v[t])) * \lambda_{\hat{j}}(v[t])$ 
vector<double> scdmembre(double (*rhsf)(double, double), int
    N, int M, double a, double b)
{
    // table de connectivite
    vector<vector<int>> TRG = maillageTR(N, M);
    // G = nombre total de noeuds
    int G = (N+1)*(M+1);
    // K = nombre total de triangles
    int K = 2*N*M;
    // On initialise B de taille G avec que des zeros
    vector<double> B(G, 0.0);
    // subdivisions en x et y
    vector<double> sub_a = Subdiv(a, N);
    vector<double> sub_b = Subdiv(b, M);
    // coordonnees de tous les noeuds
    vector<double> xs(G), ys(G);
    for (int j = 0; j <= M; j++)
        for (int i = 0; i <= N; i++) {
            int s = numgb(N, M, i, j);
            xs[s] = sub_a[i]; // coordonnee x du noeud s
            ys[s] = sub_b[j]; // coordonnee y du noeud s
        }
    // points de quadrature sur  $T_{\hat{}}$  (points milieux des
    // cotes)
    vector<vector<double>> v =
        {{0.5,0.0},{0.0,0.5},{0.5,0.5}};
    // poids de quadrature (tous egaux a 1/6)
    vector<double> w = {1.0/6, 1.0/6, 1.0/6};
    // coordonnees locales du triangle courant
    vector<vector<double>> coord(2, vector<double>(3));
    // matrice  $B_T$  du triangle courant
    vector<vector<double>> BT;
    // indice global du sommet courant
    int s;
    // contribution du triangle au noeud, valeur de f, valeur
    // de  $\lambda_{\hat{j}}$ 
    double contrib, f_val, lambda_j;
    // boucle sur tous les triangles
    for (int i = 0; i < K; i++) {
        // indices globaux des 3 sommets du triangle i
        int n0 = TRG[i][0], n1 = TRG[i][1], n2 = TRG[i][2];
        // coordonnees x des 3 sommets

```

```

coord[0] = {xs[n0], xs[n1], xs[n2]};
// coordonnees y des 3 sommets
coord[1] = {ys[n0], ys[n1], ys[n2]};
// calcul de B_T pour ce triangle
BT = CalcMatBT(coord[0], coord[1]);
// |det(B_T)| = 2 * aire du triangle
double det = abs(BT[0][0]*BT[1][1] - BT[0][1]*BT
    [1][0]);
// boucle sur les 3 sommets du triangle
for (int j = 0; j < 3; j++) {
    // numero global du sommet j
    s = TRG[i][j];
    // initialisation de la contribution de ce
        triangle a B[s]
    contrib = 0.0;
    // boucle sur les 3 points de quadrature
    for (int t = 0; t < 3; t++) {
        // transformation du point de quadrature :
            F_T(v[t])
        double xq = coord[0][0] + BT[0][0]*v[t][0] +
            BT[0][1]*v[t][1];
        double yq = coord[1][0] + BT[1][0]*v[t][0] +
            BT[1][1]*v[t][1];
        // evaluation de f au point transforme
        f_val = rhsf(xq, yq);
        // evaluation de lambda_hat_j au point de
            quadrature v[t]
        // depend du sommet local j (pas du point de
            quadrature t)
        if (j == 0) lambda_j = 1.0 - v[t][0] - v[t
            ][1]; // lambda_hat_0
        else if (j == 1) lambda_j = v[t][0];
            // lambda_hat_1
        else
            lambda_j = v[t][1];
            // lambda_hat_2
        // accumulation : poids * f * lambda_hat_j
        contrib += w[t] * f_val * lambda_j;
    }
    // multiplication par |det(B_T)| et accumulation
        dans B[s]
    B[s] += det * contrib;
}
}
return B;
}
// Norme L2 de v : sqrt(V^t * A * V) avec eta=0, lambda=1
// On utilise seulement ReacTerm (pas de DiffTerm)
double normL2(vector<double> V, int N, int M, double a,
    double b) {
    // table de connectivite

```

```

vector<vector<int>> TRG = maillageTR(N, M);
// nombre de noeuds et de triangles
int G = (N+1)*(M+1);
int K = 2*M*N;
// subdivisions
vector<double> sub_a = Subdiv(a, N);
vector<double> sub_b = Subdiv(b, M);
// coordonnees de tous les noeuds
vector<double> xs(G), ys(G);
for (int jj = 0; jj <= M; jj++)
    for (int ii = 0; ii <= N; ii++) {
        int s = numgb(N, M, ii, jj);
        xs[s] = sub_a[ii];
        ys[s] = sub_b[jj];
    }
// vecteur resultat W = A*V initialise a zero
vector<double> W(G, 0.0);
// coordonnees locales et matrice de reaction
vector<vector<double>> coord(2, vector<double>(3));
vector<vector<double>> reac(3, vector<double>(3));
int k, j;
double res, PROD2;
// boucle sur les triangles (analogue a matvec avec eta
=0, lambda=1)
for (int t = 0; t < K; t++) {
    // indices globaux des 3 sommets
    int n0 = TRG[t][0], n1 = TRG[t][1], n2 = TRG[t][2];
    // coordonnees des sommets
    coord[0] = {xs[n0], xs[n1], xs[n2]};
    coord[1] = {ys[n0], ys[n1], ys[n2]};
    // seulement ReactTerm : eta=0 donc DiffTerm supprime
    reac = ReactTerm(coord[0], coord[1]);
    // boucle sur les sommets locaux i
    for (int i = 0; i < 3; i++) {
        // numero global du sommet i
        k = TRG[t][i];
        res = 0.0;
        // boucle sur les sommets locaux r
        for (int r = 0; r < 3; r++) {
            // numero global du sommet r
            j = TRG[t][r];
            // PROD2 = reac[i][r] car lambda=1 et pas de
            // diffusion
            PROD2 = reac[i][r];
            res = res + V[j] * PROD2;
        } // fin boucle r
        // accumulation dans W[k]
        W[k] = W[k] + res;
    } // fin boucle i
} // fin boucle t

```

```

// ||v||^2_L2 = V^t * W, on retourne la racine
double norme2 = 0.0;
for (int k = 0; k < G; k++)
    norme2 += V[k] * W[k];

return sqrt(norme2);
}
// Norme H1 semi-normee de v : sqrt(V^t * A * V) avec eta=1,
// lambda=0
// On utilise seulement DiffTerm (pas de ReacTerm)
double normL2Grad(vector<double> V, int N, int M, double a,
double b) {
    // table de connectivite
    vector<vector<int>> TRG = maillageTR(N, M);
    // nombre de noeuds et de triangles
    int G = (N+1)*(M+1);
    int K = 2*M*N;
    // subdivisions
    vector<double> sub_a = Subdiv(a, N);
    vector<double> sub_b = Subdiv(b, M);
    // coordonnees de tous les noeuds
    vector<double> xs(G), ys(G);
    for (int jj = 0; jj <= M; jj++)
        for (int ii = 0; ii <= N; ii++) {
            int s = numgb(N, M, ii, jj);
            xs[s] = sub_a[ii];
            ys[s] = sub_b[jj];
        }
    // ET[t] = |det(BT)|/2 car eta=1 donc int_T eta = aire(T)
    vector<double> ET(K);
    for (int t = 0; t < K; t++) {
        int n0=TRG[t][0], n1=TRG[t][1], n2=TRG[t][2];
        double x0=xs[n0],y0=ys[n0];
        double x1=xs[n1],y1=ys[n1];
        double x2=xs[n2],y2=ys[n2];
        // |det(BT)| = 2*aire => int_T 1 = |det|/2
        ET[t] = abs((x1-x0)*(y2-y0)-(x2-x0)*(y1-y0)) / 2.0;
    }
    // vecteur resultat W = A*V initialise a zero
    vector<double> W(G, 0.0);
    // coordonnees locales et matrice de diffusion
    vector<vector<double>> coord(2, vector<double>(3));
    vector<vector<double>> diff(3, vector<double>(3));
    int k, j;
    double res, PROD2;
    // boucle sur les triangles (analogue a matvec avec eta
    // =1, lambda=0)
    for (int t = 0; t < K; t++) {
        // indices globaux des 3 sommets
        int n0 = TRG[t][0], n1 = TRG[t][1], n2 = TRG[t][2];

```

```

// coordonnees des sommets
coord[0] = {xs[n0], xs[n1], xs[n2]};
coord[1] = {ys[n0], ys[n1], ys[n2]};
// seulement DiffTerm avec ET[t] = aire(T) car eta=1
diff = DiffTerm(coord[0], coord[1], ET[t]);
// boucle sur les sommets locaux i
for (int i = 0; i < 3; i++) {
    // numero global du sommet i
    k = TRG[t][i];
    res = 0.0;
    // boucle sur les sommets locaux r
    for (int r = 0; r < 3; r++) {
        // numero global du sommet r
        j = TRG[t][r];
        // PROD2 = diff[i][r] car lambda=0 (pas de
        // ReacTerm)
        PROD2 = diff[i][r];
        res = res + V[j] * PROD2;
    } // fin boucle r
    // accumulation dans W[k]
    W[k] = W[k] + res;
} // fin boucle i
} // fin boucle t
// ||grad v||^2_L2 = V^t * W, on retourne la racine
double norme2 = 0.0;
for (int k = 0; k < G; k++)
    norme2 += V[k] * W[k];

return sqrt(norme2);
}
// Produit scalaire euclidien de deux vecteurs
double pdt_scal(vector<double> u, vector<double> v) {
    double res = 0.0; // initialise a zero
    for (int i = 0; i < u.size(); i++)
        res += u[i] * v[i]; // somme des produits terme a
        terme
    return res;
}
// Norme euclidienne d'un vecteur
double norme_vect(vector<double> u) {
    return sqrt(pdt_scal(u, u)); // racine du produit
    scalaire avec lui-meme
}
// methode du gradient biconjugué stabilise BICGSTAB pour
resoudre A*X = B
vector<double> bicgstab(
    vector<double> B,
    int N, int M,
    double a, double b,
    double lambda,

```

```

double (*eta_coef)(double, double))
{
    // tolerance de convergence
    double tol= 1e-6;
    // nombre max d'iterations
    int itermax = 1000;
    // taille du systeme
    int n = B.size();
    // X0 = vecteur nul (solution initiale)
    vector<double> X(n, 0.0);
    // R0 = B - A*X0 = B (car X0 = 0)
    vector<double> R = B;
    // R0* = R0 (choix arbitraire)
    vector<double> Rx = R;
    // W0 = R0
    vector<double> W = R;
    // vecteurs intermediaires de l'algorithme
    vector<double> AW(n), Sj(n), ASj(n), Rj(n);
    // scalaires de l'algorithme
    double alpha, omega, beta;
    // convergence initiale = norme du residu initial
    double conv = norme_vect(R);
    // compteur d'iterations
    int j = 0;
    // boucle jusqu'a convergence ou nombre max d'iterations
    while (j < itermax && conv > tol) {
        // AW_j = A * W_j
        AW = matvec(W, N, M, a, b, lambda, eta_coef);
        // alpha_j = <R_j, R0*> / <AW_j, R0*>
        alpha = pdt_scal(R, Rx) / pdt_scal(AW, Rx);
        // S_j = R_j - alpha_j * AW_j
        for (int i =0; i < n; i++)
            Sj[i] = R[i] - alpha * AW[i];
        // AS_j = A * S_j
        ASj = matvec(Sj, N, M, a, b, lambda, eta_coef);
        // omega_j = <AS_j, S_j> / <AS_j, AS_j>
        omega = pdt_scal(ASj, Sj) / pdt_scal(ASj, ASj);
        // X_{j+1} = X_j + alpha_j * W_j + omega_j * S_j
        for (int i = 0; i < n; i++)
            X[i] = X[i] + alpha * W[i] + omega * Sj[i];
        // R_{j+1} = S_j - omega_j * AS_j
        for (int i = 0; i < n; i++)
            Rj[i] = Sj[i] - omega * ASj[i];
        // beta_j = (<R_{j+1}, R0*>/<R_j, R0*>) * (alpha_j/
            omega_j)
        beta = (pdt_scal(Rj, Rx) / pdt_scal(R, Rx)) * (alpha
            / omega);
        // W_{j+1} = R_{j+1} + beta_j * (W_j - omega_j * AW_j
            )
        for (int i = 0; i < n; i++)

```

```

        W[i] = Rj[i] + beta * (W[i] - omega * AW[i]);
// mise a jour du residu et du compteur
R = Rj;
conv = norme_vect(R);
j++;
}
return X;
}
// Constante pi
const double pi = 3.141592653589793;
// Parametre m global (entier naturel de l'enonce)
int m_global = 1;
// Solution exacte  $u^p(x,y) = \cos(m\pi x)\cos(m\pi y)/(2m^2\pi^2+1)$ 
double solExa(double x, double y) {
    return cos(m_global * pi * x) * cos(m_global * pi * y)
        / (2.0 * m_global*m_global * pi*pi + 1.0);
}
// Second membre  $f(x,y) = \cos(m\pi x)\cos(m\pi y)$ 
double rhsf(double x, double y) {
    return cos(m_global * pi * x) * cos(m_global * pi * y);
}
// Calcule les 3 erreurs relatives entre  $u^h$  et  $u^p$ 
// e0 =  $\|u^h - u^p\|_{L2} / \|u^h\|_{L2}$ 
// e1 =  $\|\text{grad}(u^h - u^p)\|_{L2} / \|\text{grad}(u^h)\|_{L2}$ 
// einf =  $\max|u^h - u^p| / \max|u^h|$ 
vector<double> erreurs(
    double (*solExa)(double, double),
    vector<double> uh, // solution approchee de taille G
    int N, int M,
    double a, double b)
{
    // nombre total de noeuds
    int G = (N+1)*(M+1);
    // subdivisions
    vector<double> sub_a = Subdiv(a, N);
    vector<double> sub_b = Subdiv(b, M);
    // coordonnees de tous les noeuds
    vector<double> xs(G), ys(G);
    for (int j = 0; j <= M; j++)
        for (int i = 0; i <= N; i++) {
            int s = numgb(N, M, i, j);
            xs[s] = sub_a[i];
            ys[s] = sub_b[j];
        }
    // uh_hat = interpolée de  $u^p$  aux noeuds du maillage
    vector<double> uh_hat(G);
    for (int k = 0; k < G; k++)
        uh_hat[k] = solExa(xs[k], ys[k]);
    // diff_vec = uh_hat - uh (difference noeud par noeud)

```



```

vector<double> diff_vec(G);
for (int k = 0; k < G; k++)
    diff_vec[k] = uh_hat[k] - uh[k];
// e0 = erreur relative en norme L2
double e0 = normL2(diff_vec, N, M, a, b)
           / normL2(uh_hat, N, M, a, b);
// e1 = erreur relative en semi-norme H1 (gradient)
double e1 = normL2Grad(diff_vec, N, M, a, b)
           / normL2Grad(uh_hat, N, M, a, b);
// calcul de l'erreur infinie
double maxdiff = 0.0; // max de |uh_hat - uh|
double maxw     = 0.0; // max de |uh_hat|

for (int k = 0; k < G; k++) {
    // mise a jour du max de l'erreur absolue
    if (abs(diff_vec[k]) > maxdiff)
        maxdiff = abs(diff_vec[k]);
    // mise a jour du max de la solution exacte
    if (abs(uh_hat[k]) > maxw)
        maxw = abs(uh_hat[k]);
}
// einf = erreur relative en norme infinie
double einf = maxdiff / maxw;
// retourne les 3 erreurs
return {e0, e1, einf};
}
int main() {

    double a      = 1.0;
    double b      = 1.0;
    double lambda = 1.0;
    m_global      = 1;    // m fixe a 1

    // En-tete du tableau
    cout << setw(5) << "N"
         << setw(12) << "h"
         << setw(12) << "e0"
         << setw(12) << "e1"
         << setw(12) << "einf" << "\n";
    cout << string(53, '-') << "\n";

    // Boucle sur N croissant (h decroissant)
    for (int N : {5, 10, 20, 40, 80}) {

        int M = N;
        double h = 2.0 * a / N; // pas du maillage

        // Second membre B
        vector<double> B = scdmembre(rhsf, N, M, a, b);

```

```

// Resolution par BICGSTAB
// eta_coef doit renvoyer 1.0 car eta0 = 0
vector<double> uh = bicgstab(B, N, M, a, b,
                             lambda, eta_coef);

// Calcul des erreurs
vector<double> err = erreurs(solExa, uh, N, M, a, b);

cout << setw(5) << N
      << setw(12) << scientific
      << setprecision(3) << h
      << setw(12) << err[0]
      << setw(12) << err[1]
      << setw(12) << err[2] << "\n";
}

return 0;
}

```